МИНОБРНАУКИ РОССИИ САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА

Кафедра вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

По теме: «Управление файловой системой»

Студент гр. 4312:

Устинов В. А.

Преподаватель:

Тимофеев А. В.

Санкт-Петербург

2026

Цель работы

Исследовать управление файловой системой с помощьюWin32API.

Задание 1.1. Управление дисками, каталогами и файлами

В ходе первой части лабораторной работы было создано консольное приложение, предназначенное для исследования и управления файловой системой Windows. Для разработки приложения потребовались стандартные C++ библиотеки и набор базовых функций интерфейсов программирования приложений операционных систем семейств Microsoft Windows. Функционал приложения будет описан ниже по пунктам.

```
            Menu:
(0)  Exit;
(1)  Output list of disks;
(2)  Output list of disks;
(3)  Driver Type;
(4)  Volume Information;
(5)  Disk free space;
(6)  Create directory;
(7)  Remove directory;
(8)  Create file in directory;
(9)  Copy file;
(10) Move file;
(11) Move file (extended);
(12) File attributes;
(13) Set attributes;
(14) Get file info;
(15) Get file time;
(16) Set file time;

Choose function:
```

Рис. 1 — Главное меню программы

1) Вывод списка дисков (через битовую маску)

Функция ***showLogicalDrives*** выводит список дисков, которые установлены в системе в текущий момент. Главной частью функции является ***GetLogicalDrives***, возвращающая битовую

маску типа DWORD (32-битное беззнаковое целое), где каждый бит соответствует наличию диска.

Было проведено тестирование функции, в первом случае выведены диски моего компьютера (см Рис.2), во втором случае выведены диски и флешка (см Рис. 3).

```
            Menu:
(0)  Exit;
(1)  Output list of disks;
(2)  Output list of disks;
(3)  Driver Type;
(4)  Volume Information;
(5)  Disk free space;
(6)  Create directory;
(7)  Remove directory;
(8)  Create file in directory;
(9)  Copy file;
(10) Move file;
(11) Move file (extended);
(12) File attributes;
(13) Set attributes;
(14) Get file info;
(15) Get file time;
(16) Set file time;

Choose function: 1

Your drives:
1. C:\
2. D:\
```

Рис. 2

```
            Menu:
(0)  Exit;
(1)  Output list of disks;
(2)  Output list of disks;
(3)  Driver Type;
(4)  Volume Information;
(5)  Disk free space;
(6)  Create directory;
(7)  Remove directory;
(8)  Create file in directory;
(9)  Copy file;
(10) Move file;
(11) Move file (extended);
(12) File attributes;
(13) Set attributes;
(14) Get file info;
(15) Get file time;
(16) Set file time;

Choose function: 1

Your drives:
1. C:\
2. D:\
3. E:\
```

Рис. 3

2) Вывод списка дисков (через список строк)

Функция ***showLogicalDrivesSTR*** аналогично выводит список дисков, которые установлены в системе в текущий момент. Главной частью функции является ***GetLogicalDriveStringsA*** заполняющая буфер в многострочном формате с двойным нуль-терминатором.

Были проведены аналогичные испытания. На Рис. 4 флешка и диски системы, на Рис. 5 только диски системы.

```
            Menu:
(0)  Exit;
(1)  Output list of disks;
(2)  Output list of disks;
(3)  Driver Type;
(4)  Volume Information;
(5)  Disk free space;
(6)  Create directory;
(7)  Remove directory;
(8)  Create file in directory;
(9)  Copy file;
(10) Move file;
(11) Move file (extended);
(12) File attributes;
(13) Set attributes;
(14) Get file info;
(15) Get file time;
(16) Set file time;

Choose function: 2

Your drives:
1. C:\
2. D:\
3. E:\
```

Рис. 4

```
            Menu:
(0)  Exit;
(1)  Output list of disks;
(2)  Output list of disks;
(3)  Driver Type;
(4)  Volume Information;
(5)  Disk free space;
(6)  Create directory;
(7)  Remove directory;
(8)  Create file in directory;
(9)  Copy file;
(10) Move file;
(11) Move file (extended);
(12) File attributes;
(13) Set attributes;
(14) Get file info;
(15) Get file time;
(16) Set file time;

Choose function: 2

Your drives:
1. C:\
2. D:\
```

Рис. 5

3) Вывод типа диска

Функция ***showDriveType*** выводит тип носителя, для выбранного пользователем диска. Главная функция программы ***GetDriveTypeA*** возвращающая значение UINT (беззнаковое целое), определяющее тип носителя(

Код	Значение	Описание
0	DRIVE_UNKNOWN	Тип диска не может быть определен
1	DRIVE_NO_ROOT_DIR	Корневой путь неверен
2	DRIVE_REMOVABLE	Съемный диск
3	DRIVE_FIXED	Фиксированный диск (HDD, SSD)
4	DRIVE_REMOTE	Сетевой диск
5	DRIVE_CDROM	CD/DVD/Blu-ray привод

Были проведены три определения типов носителей:

```
Choose function: 3

Please select a disk number: 1
DRIVE_FIXED
```

Рис. 6 — C:

```
Choose function: 3

Please select a disk number: 2
DRIVE_CDROM
```

Рис. 7— D:

```
Choose function: 3

Please select a disk number: 3
DRIVE_REMOVABLE
```

Рис. 8 — E:

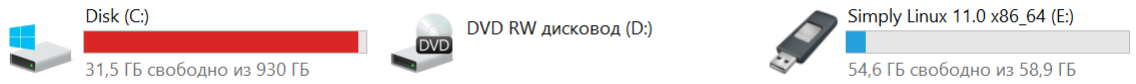


Рис. 9 — Диски моей системы

Как видно из Рис. 9 программа корректно определяет тип носителей.

4) Вывод детальной информации о носителе

Функция ***showDriverInformation*** вывод подробную информацию о носителе (имя тома, серийный номер, тип файловой системы и разрешения диска). Информация получается через функцию ***GetVolumeInformationA.***

Проведено исследование моего жесткого диска (см Рис. 10). По выведенной информации можно узнать основные данные, а так же и детальные.

```
Choose function: 4

Please select a disk number: 1
Disk name: Disk
Serial number: 440854531
File system: NTFS
Max length of file name component: 255
File system info:
  (+) Case-sensitive search
  (+) Preserves case of filenames
  (+) Unicode filenames
  (+) File-based compression
  (+) Disk quotas
  (+) Sparse files
  (+) Reparse points
  (+) Object identifiers
  (+) Encrypted File System (EFS)
  (+) Named streams
  (+) Transactions
  (+) Hard links
  (+) Extended attributes
```

Рис. 10

Флаг	Значение	Бит	Описание
FILE_CASE_SENSITIVE_SEARCH	0x00000001	0	Файловая система чувствительна к регистру при поиске
FILE_CASE_PRESERVED_NAMES	0x00000002	1	Сохраняет регистр имен файлов
FILE_UNICODE_ON_DISK	0x00000004	2	Позволяет использовать кириллицу, иероглифы
FILE_FILE_COMPRESSION	0x00000010	4	Поддерживает сжатие отдельных файлов

FILE_VOLUME_QUOTAS	0x00000020	5	Ограничение места для пользователей
FILE_SUPPORTS_SPARSE_FILES	0x00000040	6	Поддерживает разреженные файлы
FILE_VOLUME_IS_COMPRESSED	0x00008000	15	Весь том сжат
FILE_SUPPORTS_ENCRYPTION	0x00020000	17	Поддерживает шифрование
FILE_NAMED_STREAMS	0x00040000	18	Поддерживает именованные потоки
FILE_READ_ONLY_VOLUME	0x00080000	19	Том только для чтения
FILE_SUPPORTS_TRANSACTIONS	0x00200000	21	Поддерживает транзакции
FILE_SUPPORTS_HARD_LINKS	0x00400000	22	Поддерживает жесткие ссылки

5) Вывод секторов диска и его свободного места

Функция ***showDriverFreeSpace*** выводит подробную информацию о памяти диска, о его кластерах и секторах в них. Эта информация получается благодаря функции ***GetDiskFreeSpaceA***. Ниже представлен вывод моей функции и сравнение с информацией проводника (см Рис. 11-13).

```
Choose function: 5

Please select a disk number: 1
(1) Sectors per cluster: 8
(2) Bytes per sector: 512
(3) Number of free clusters: 4516035
(4) Total clusters: 244023595
(5) Total disk space: 953217 MB
(6) Free disk space: 17640 MB
```

Рис. 11

```
Choose function: 5

Please select a disk number: 3
(1) Sectors per cluster: 64
(2) Bytes per sector: 512
(3) Number of free clusters: 1791927
(4) Total clusters: 1932287
(5) Total disk space: 60383 MB
(6) Free disk space: 55997 MB
```

Рис. 12

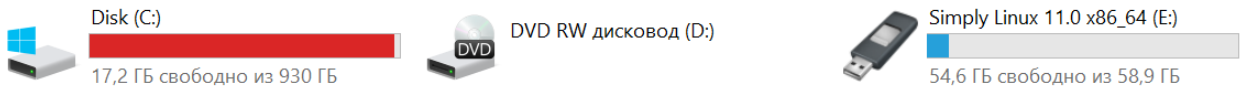
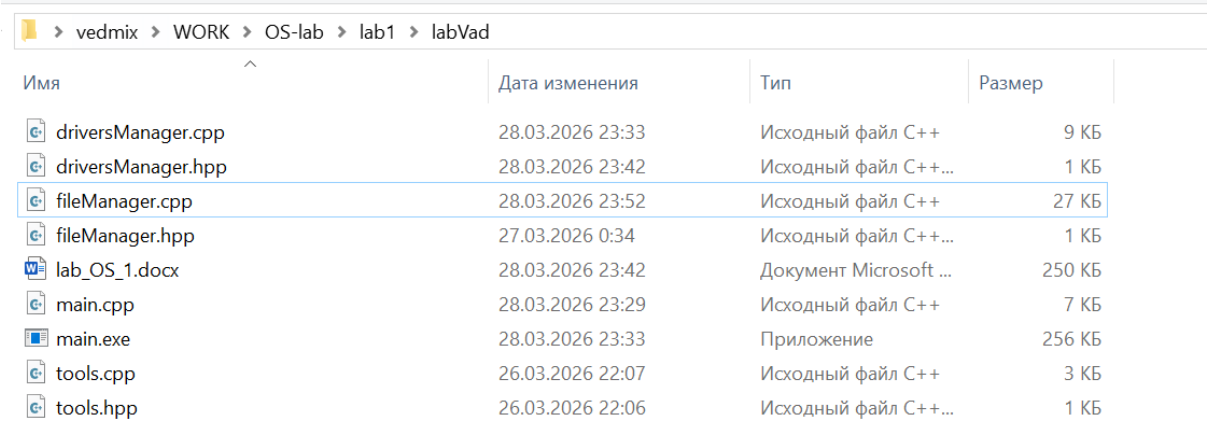


Рис. 13

6) Создание директории

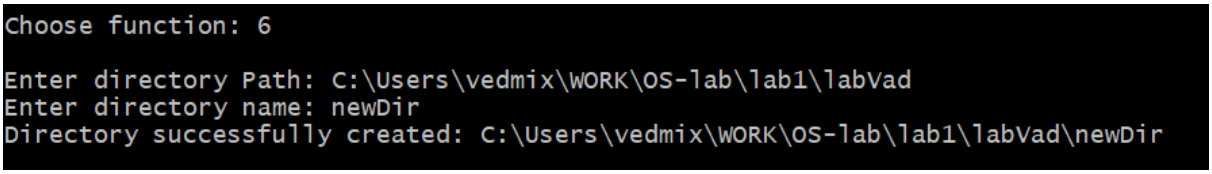
Для создания директорий я улучшил стандартную функцию Win32 API ***CreateDirectory***, добавив в нее проверки до создания и обработку исключений в ходе самого создания директории. Ниже показано, как она вызывается и работает (см Рис. 14-16).



vedmix > WORK > OS-lab > lab1 > labVad

Имя	Дата изменения	Тип	Размер
driversManager.cpp	28.03.2026 23:33	Исходный файл C++	9 КБ
driversManager.hpp	28.03.2026 23:42	Исходный файл C++...	1 КБ
fileManager.cpp	28.03.2026 23:52	Исходный файл C++	27 КБ
fileManager.hpp	27.03.2026 0:34	Исходный файл C++...	1 КБ
lab_OS_1.docx	28.03.2026 23:42	Документ Microsoft ...	250 КБ
main.cpp	28.03.2026 23:29	Исходный файл C++	7 КБ
main.exe	28.03.2026 23:33	Приложение	256 КБ
tools.cpp	26.03.2026 22:07	Исходный файл C++	3 КБ
tools.hpp	26.03.2026 22:06	Исходный файл C++...	1 КБ

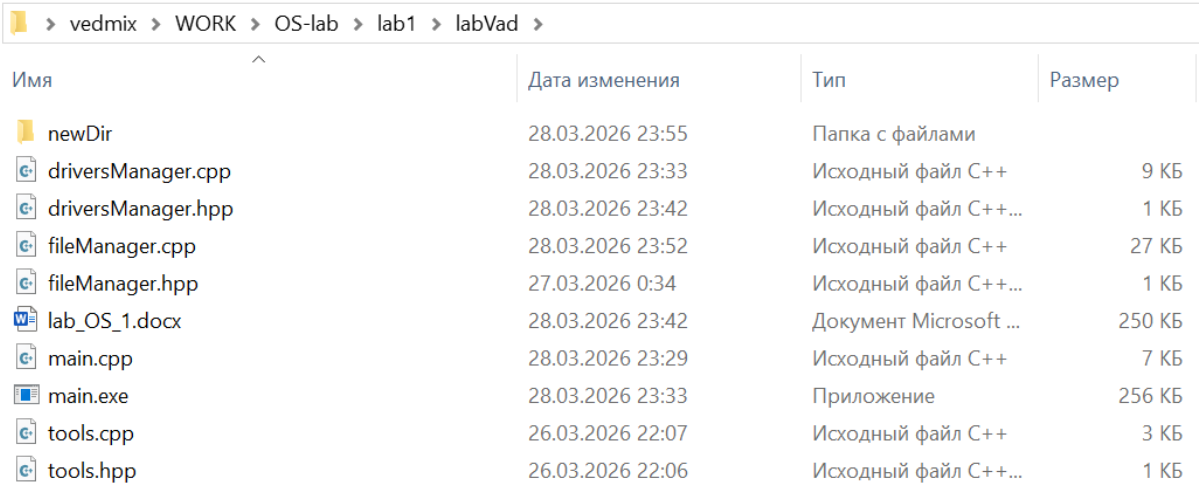
Рис. 14



```
Choose function: 6

Enter directory Path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad
Enter directory name: newDir
Directory successfully created: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newDir
```

Рис. 15



vedmix > WORK > OS-lab > lab1 > labVad >

Имя	Дата изменения	Тип	Размер
newDir	28.03.2026 23:55	Папка с файлами	
driversManager.cpp	28.03.2026 23:33	Исходный файл C++	9 КБ
driversManager.hpp	28.03.2026 23:42	Исходный файл C++...	1 КБ
fileManager.cpp	28.03.2026 23:52	Исходный файл C++	27 КБ
fileManager.hpp	27.03.2026 0:34	Исходный файл C++...	1 КБ
lab_OS_1.docx	28.03.2026 23:42	Документ Microsoft ...	250 КБ
main.cpp	28.03.2026 23:29	Исходный файл C++	7 КБ
main.exe	28.03.2026 23:33	Приложение	256 КБ
tools.cpp	26.03.2026 22:07	Исходный файл C++	3 КБ
tools.hpp	26.03.2026 22:06	Исходный файл C++...	1 КБ

Рис. 16

Из скриншотов видно, что все корректно создается, если указан правильный родительский путь.

```
Choose function: 6

Enter directory Path:  C:\Users\vedmix\WORK\OS-lab\lab1\labVad\VOID
Error: Parent directory does not exist:  C:\Users\vedmix\WORK\OS-lab\lab1\labVad\VOID
```

Рис. 17

При несуществующей родительской директории программа выдает ошибку (см Рис. 17).

7) Удаление директории

Аналогично созданию используется встроенная функция ***RemoveDirectory*** с добавленными проверками и обработкой исключений. Ниже представлены различные случаи удаления директории (правильное, удаление несуществующей директории и удаление не пустой директории см Рис. 18 – 20).

```
Choose function: 7

Enter directory Path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newDir
Removing directory: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newDir
Directory successfully removed: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newDir
```

Рис. 18

```
Choose function: 7

Enter directory Path:  C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newDir
Removing directory: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newDir
Error: Directory does not exist!
```

Рис. 19

```
Choose function: 7

Enter directory Path:  C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dirNotEmpty
Removing directory: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dirNotEmpty
Error: Directory is not empty! Please delete all files first.
```

Рис. 20

Из скриншотов видно, что функция корректно ловит и обрабатывает ошибки и исключительные ситуации.

8) Создание файла в директории

Создание файла происходит через функцию ***CreateFile***. В функции добавлены проверки директории и обработка исключительных ситуаций. Добавить в директорию можно файл с любым расширением, расширение вводится пользователем при создании файла. Ниже представлены скриншоты, демонстрирующие работу функции (см Рис. 21-24).

```
Choose function: 8

Enter directory path:  C:\Users\vedmix\WORK\OS-lab\lab1\labVad
Enter file name: newFile.txt
Enter file content (Enter for empty):
Creating file: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newFile.txt
File successfully created: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newFile.txt
```

Рис. 21

```
Choose function: 8

Enter directory path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad
Enter file name: text.txt
Enter file content (Enter for empty): hello world!
Creating file: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\text.txt
Written 12 bytes to file
File successfully created: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\text.txt
```

Рис. 22

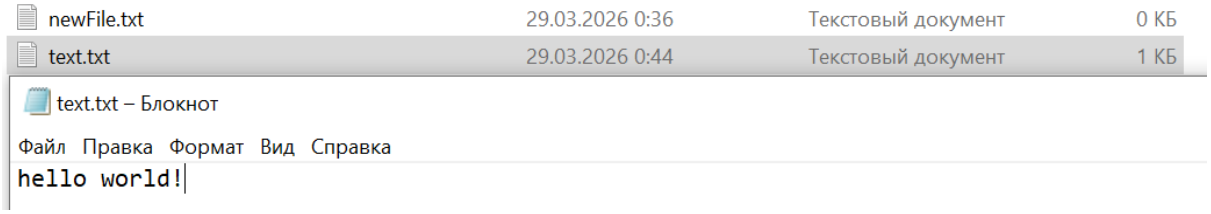


Рис. 23

```
Choose function: 8

Enter directory path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\jjj
Error: directory does not exist: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\jjj
```

Рис. 24

Из иллюстраций видно, что файлы создаются, и по желанию пользователя в них можно записывать разные данные, исключительные ситуации обрабатываются.

9) Копирование файла в выбранную директорию

Копирование файла происходит с помощью функции ***CopyFile*** из Win32 API и добавленными обработчиками исключительных ситуаций и ошибок. Программа запрашивает, полный путь к файлу, который нужно скопировать, путь куда нужно скопировать, если копия уже есть, программа предложит ее перезаписать. Ниже показана работа функции + исключительные ситуации(см Рис. 25-29).

```
Choose function: 9

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newFile.txt
Enter destination directory: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir
File successfully copied!
```

Рис. 25

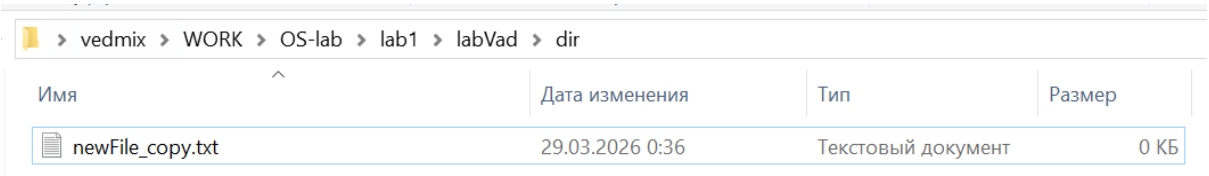


Рис. 26

```
Choose function: 9

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newFile.txt
Enter destination directory: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir
File already exists! Overwrite? (y/n): y
File successfully copied!
```

Рис. 27

```
Choose function: 9

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\newFile.txt
Enter destination directory: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\fake
Error: directory C:\Users\vedmix\WORK\OS-lab\lab1\labVad\fake is incorrect
```

Рис. 28

```
Choose function: 9

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\txt.t
Error: directory C:\Users\vedmix\WORK\OS-lab\lab1\labVad\txt.t is incorrect
```

Рис. 29

Как видно из скриншотов, операция выполняется правильно, исключительные ситуации обрабатываются.

10) Перемещение файла в файловой системе

Операция позволяет переместить, файл в директорию или переместить содержимое одного файла в другой, с помощью ***MoveFile*** Win32 API. Ниже представлены рабочие случаи + исключения(см Рис. 30-33).

```
Choose function: 10

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
Enter destination: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir
File successfully moved!
From: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
To: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
```

Рис. 30

```
Choose function: 10

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
Enter destination: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file1.txt
File successfully moved!
From: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
To: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file1.txt
```

Рис. 31

```
Choose function: 10

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
Enter destination: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
Destination file already exists!
Overwrite? (y/n): y
File successfully moved!
From: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
To: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
```

Рис. 32

```
Choose function: 10

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
Enter destination: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir1\
Error: destination directory does not exist!
```

Рис. 33

Функция работает стабильно, главная ее проблема — это отсутствие особых прав, для чего нужен следующий пункт.

11) Перемещение файла с особыми правами

Перемещение файла с флагами производится с помощью функции ***MoveFileEx*** из Win32 API, GUI с выбором флагов и обработкой исключений. После ввода корректных путей, программа предлагает пользователю выбрать флаг для перемещения (см Рис. 34).

```
Select MoveFileEx flags:
(1) Normal move;
(2) Replace existing file;
(3) Allow copy between different drives;
```

Рис. 34

Флаг	**Описание**
MOVEFILE_REPLACE_EXISTING	Перезаписывает файл, если он уже существует в месте назначения.
MOVEFILE_COPY_ALLOWED	Разрешает перемещение между разными дисками
MOVEFILE_WRITE_THROUGH	Гарантирует завершение операции до возврата функции

Ниже представлены иллюстрации работы функции (см Рис. 35-38).

```
Choose function: 11

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
Enter destination: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\

Select MoveFileEx flags:
(1) Normal move;
(2) Replace existing file;
(3) Allow copy between different drives;

Choose flag option: 1
Move successful
From: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
To:   C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
```

Рис. 35

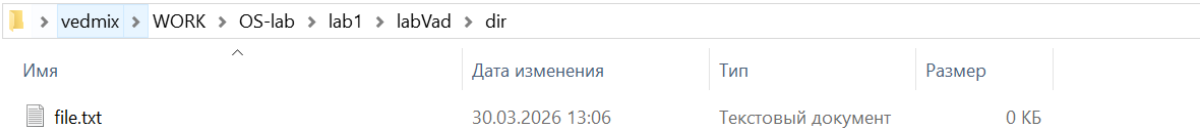


Рис. 36

```
Choose function: 11

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
Enter destination: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\

Select MoveFileEx flags:
(1) Normal move;
(2) Replace existing file;
(3) Allow copy between different drives;

Choose flag option: 2
Move successful
From: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\file.txt
To:   C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
```

Рис. 37

```
Choose function: 11

Enter source file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
Enter destination: E:\

Select MoveFileEx flags:
(1) Normal move;
(2) Replace existing file;
(3) Allow copy between different drives;

Choose flag option: 3
Move successful
From: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\file.txt
To:   E:\file.txt
```

Рис. 38

Расширенная функция работает стабильно и обрабатывает ошибки, в отличие от обычной функции MoveFile, расширенной требуется права администратора для корректной работы.

12) Вывод атрибутов файла

Вывод атрибутов реализован в функции ***showFileAttributes***, которая использует ***GetFileAttributesA*** из Win32 API и обрабатывает исключительные ситуации. Функция ***GetFileAttributesA*** возвращает атрибуты, которые описаны ниже.

Атрибут	**Значение**	**Описание**
FILE_ATTRIBUTE_READONLY	0x01	Только для чтения
FILE_ATTRIBUTE_HIDDEN	0x02	Файл не отображается в проводнике по умолчанию
FILE_ATTRIBUTE_SYSTEM	0x04	Файл используется операционной системой
FILE_ATTRIBUTE_DIRECTORY	0x10	Директория
FILE_ATTRIBUTE_ARCHIVE	0x20	Файл изменён после последнего резервного копирования
FILE_ATTRIBUTE_NORMAL	0x80	Нет других атрибутов
FILE_ATTRIBUTE_TEMPORARY	0x100	Временный файл
FILE_ATTRIBUTE_COMPRESSED	0x800	Сжатый, файл сжат NTFS-сжатием
FILE_ATTRIBUTE_ENCRYPTED	0x4000	Зашифрованный. Файл зашифрован с помощью EFS

Для проверки функции, я создал скрытый txt файл и файл с ограничением права на запись. Результаты проверки см Рис. 39-42.

```
Choose function: 12

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\dir\":
 Directory
Attribute value (hex): 0x10
```

Рис. 39

```
Choose function: 12

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt":
 Read-only
 Archive
Attribute value (hex): 0x21
```

Рис. 40

```
Choose function: 12

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\hidden.txt
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\hidden.txt":
 Hidden
 Archive
Attribute value (hex): 0x22
```

Рис. 41

```
Choose function: 12

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Archive
Attribute value (hex): 0x20
```

Рис. 42

Как видно из скриншотов атрибуты считываются корректно.

13) Назначение атрибутов

Атрибуты назначаются файлам через сеттер ***setFileAttribute***, он использует функцию API ***SetFileAttributesA*** и умеет удобный интерфейс для выбора флагов атрибутов. Ниже представлены иллюстрации работы (см Рис. 43-46)

```
Choose function: 13

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt
Current attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Archive
Attribute value (hex): 0x20

Choose attribute to set:
(1)  ARCHIVE
(2)  HIDDEN
(3)  NORMAL (clears all other attributes)
(4)  NOT CONTENT INDEXED
(5)  ENCRYPTED
(6)  READONLY
(7)  SYSTEM
(8)  TEMPORARY
(9)  COMPRESSED
Choose attribute: 2
Attributes successfully changed!
New attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Hidden
Attribute value (hex): 0x2
```

Рис. 43

```
Choose function: 13

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt
Current attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Hidden
Attribute value (hex): 0x2

Choose attribute to set:
(1)  ARCHIVE
(2)  HIDDEN
(3)  NORMAL (clears all other attributes)
(4)  NOT CONTENT INDEXED
(5)  ENCRYPTED
(6)  READONLY
(7)  SYSTEM
(8)  TEMPORARY
(9)  COMPRESSED
Choose attribute: 3
Attributes successfully changed!
New attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Normal file
Attribute value (hex): 0x80
```

Рис. 44

```
Choose function: 13

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt
Current attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Normal file
Attribute value (hex): 0x80

Choose attribute to set:
(1)  ARCHIVE
(2)  HIDDEN
(3)  NORMAL (clears all other attributes)
(4)  NOT CONTENT INDEXED
(5)  ENCRYPTED
(6)  READONLY
(7)  SYSTEM
(8)  TEMPORARY
(9)  COMPRESSED
Choose attribute: 6
Attributes successfully changed!
New attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Read-only
Attribute value (hex): 0x1
```

Рис. 45

```
Choose function: 13

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt
Current attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Read-only
Attribute value (hex): 0x1

Choose attribute to set:
(1)  ARCHIVE
(2)  HIDDEN
(3)  NORMAL (clears all other attributes)
(4)  NOT CONTENT INDEXED
(5)  ENCRYPTED
(6)  READONLY
(7)  SYSTEM
(8)  TEMPORARY
(9)  COMPRESSED
Choose attribute: 8
Attributes successfully changed!
New attributes:
Attributes of file "C:\Users\vedmix\WORK\OS-lab\lab1\labVad\normal.txt":
 Temporary
Attribute value (hex): 0x100
```

Рис. 46

14) Вывод информации о файле

Вывод информации о файле реализован в функции ***showFileInfo*** через функцию ***GetFileInformationByHandle***. Примеры использования функции приведены ниже (см Рис 47-48).

```
Choose function: 14

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\hidden.txt

File Information:
File: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\hidden.txt
Attributes: Hidden Archive
Created: 30.03.2026 10:40:33
Last access: 30.03.2026 10:40:33
Last modified: 30.03.2026 10:40:33

Size: 5 bytes
Volume serial number: 0x1a46e803
Number of hard links: 1
Unique file ID: 0x1d00000006bd56
```

Рис. 47

```
Choose function: 14

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt

File Information:
File: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt
Attributes: Read-only Archive
Created: 30.03.2026 10:40:33
Last access: 30.03.2026 10:40:33
Last modified: 30.03.2026 10:40:33

Size: 5 bytes
Volume serial number: 0x1a46e803
Number of hard links: 1
Unique file ID: 0x2f00000006bd5b
```

Рис. 48

Функция работает корректно, выводит время изменения, создания, открытия файла, его вес и id.

15) Вывод информации о времени файла

Вывод информации производится через функцию ***showFileTimeInfo.*** Функция выводит информацию о дате создания, изменения и открытия файла. Сделано это через функцию ***GetFileTime*** из Win32 API. Работа функции представлена ниже (см Рис. 49-50).

```
Choose function: 15

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt

File Time Information:
File: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt
Created: 30.03.2026 10:40:33
Last accessed: 30.03.2026 10:40:33
Last modified: 30.03.2026 10:40:33
```

Рис. 49

```
Choose function: 15

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\main.cpp

File Time Information:
File: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\main.cpp
Created: 21.03.2026 13:58:37
Last accessed: 30.03.2026 13:21:18
Last modified: 30.03.2026 13:20:48
```

Рис. 50

Функция работает корректно.

16) Переустановка времени файла

Переустановка времени происходит через сеттер ***setFileTime***. В нем можно настроить временные характеристики файла. Поставить текущее время, поставить выбранное время, и поменять время конкретного действия над файлом. Ниже представлены примеры настройки времени (см Рис.

```
Choose function: 16

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt

=== Current File Times ===

File Time Information:
File: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt
Created: 30.03.2026 10:40:33
Last accessed: 30.03.2026 10:40:33
Last modified: 30.03.2026 10:40:33

=== Select time mode ===
(1) Set to current system time
(2) Set custom date and time
(3) Set creation time only
(4) Set last access time only
(5) Set last modified time only
Choose option: 1
Setting all file times to current system time...

File times successfully updated!

=== New File Times ===
Created: 30.03.2026 13:29:24
Last accessed: 30.03.2026 13:29:24
Last modified: 30.03.2026 13:29:24
```

Рис. 51

```
Choose function: 16

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt

=== Current File Times ===

File Time Information:
File: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt
Created: 23.03.30826 02:01:01
Last accessed: 30.03.2026 13:30:52
Last modified: 01.01.2026 01:01:01

=== Select time mode ===
(1) Set to current system time
(2) Set custom date and time
(3) Set creation time only
(4) Set last access time only
(5) Set last modified time only
Choose option: 2

Enter custom date and time:
Year (1601-30827): 1601
Month (1-12): 3
Day (1-31): 23
Hour (0-23): 5
Minute (0-59): 32
Second (0-59): 2
Setting all file times to custom date/time...

File times successfully updated!

=== New File Times ===
Created: 23.03.1601 05:32:02
Last accessed: 23.03.1601 05:32:02
Last modified: 23.03.1601 05:32:02
```

Рис. 52

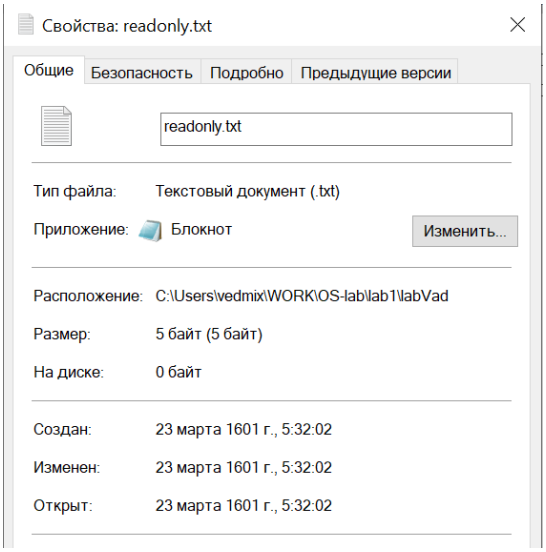


Рис. 53

```
Choose function: 16

Enter file path: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt

=== Current File Times ===

File Time Information:
File: C:\Users\vedmix\WORK\OS-lab\lab1\labVad\readonly.txt
Created: 23.03.1601 02:32:02
Last accessed: 30.03.2026 13:30:52
Last modified: 23.03.1601 02:32:02

=== Select time mode ===
(1) Set to current system time
(2) Set custom date and time
(3) Set creation time only
(4) Set last access time only
(5) Set last modified time only
Choose option: 3

Enter custom creation date and time:
Year (1601-30827): 30826
Month (1-12): 3
Day (1-31): 23
Hour (0-23): 2
Minute (0-59): 1
Second (0-59): 1
Setting creation time only...

File times successfully updated!

=== New File Times ===
Created: 23.03.30826 02:01:01
Last accessed: 30.03.2026 13:30:52
Last modified: 23.03.1601 02:32:02
```

Рис. 54

Выводы по заданию 1.1

В ходе выполнения первой части лабораторной работы было разработано консольное приложение, демонстрирующее возможности Win32 API по управлению файловой системой Windows. Реализованы функции для вывода списка дисков с использованием *GetLogicalDrives* и *GetLogicalDriveStrings*, определения типа носителя через *GetDriveType*, получения детальной информации о файловой системе *GetVolumeInformation* и свободном пространстве *GetDiskFreeSpace*. Создание и удаление каталогов выполнены с помощью *CreateDirectory* и *RemoveDirectory* с добавленными проверками существования пути и обработкой ошибок. Для работы с файлами использованы *CreateFile* для создания файлов с возможностью записи данных, *CopyFile* для копирования с опцией перезаписи существующего файла, а также *MoveFile* и *MoveFileEx* для перемещения, где расширенная версия позволяет перезаписывать файлы и перемещать их между разными дисками с помощью флагов. Анализ и изменение атрибутов реализованы через GetFileAttributes и SetFileAttributes, а получение временных меток и детальной информации о файле — через *GetFileTime, SetFileTime* и *GetFileInformationByHandle*.

В процессе разработки были выявлены следующие особенности Win32 API:

1. функция *MoveFile* не поддерживает перемещение между разными томами, для этого требуется *MoveFileEx* с флагом MOVEFILE_COPY_ALLOWED;
2. при передаче пути к директории без указания имени файла *MoveFileEx* возвращает ошибку ERROR_ALREADY_EXISTS;

3. атрибуты FILE_ATTRIBUTE_TEMPORARY и FILE_ATTRIBUTE_COMPRESSED не устанавливаются автоматически на основе расширения файла, а должны быть явно заданы;
4. для выполнения операций с флагом MOVEFILE_REPLACE_EXISTING и установки системных атрибутов требуются права администратора, иначе API возвращает ERROR_ACCESS_DENIED;
5. временные метки FILETIME имеют высокую точность до 100 наносекунд.

Все эти особенности были учтены при разработке, что позволило создать стабильное приложение, корректно обрабатывающее все исключительные ситуации.

Задание 1.2 Копирование файла с помощью операций

Для решения второго задания разработано консольное приложение на языке C++, реализующее асинхронное копирование файлов с использованием библиотеки libaio (Linux AIO), библиотеки POSIX, обхода страничного кэша и перекрывающих выполнение операций. Программа состоит из трех основных классов FileCopier, TestCopying и Interface. В таблице ниже описаны назначения классов.

Название	**Назначение**
FileCopier	Реализация асинхронного копирования файлов с использованием libaio.
TestCopying	Тестовый модуль для измерения производительности копирования.
Interface	Консольный пользовательский интерфейс.

1) Класс FileCopier

Класс FileCopier включает в себя три основных поля, которые описаны ниже в таблице.

Поле	**Тип**	**Описание**
ALIGN	size_t	Выравнивание буфера в байтах.
blockSize	size_t	Размер блока копирования.
overlapCount	int	Количество перекрывающихся операций.

Главной рабочей частью класса является функция *copyFile*, выполняющая асинхронное копирование файла из исходного пути в целевую директорию, используя прямой доступ к диску без кэширования. Ниже описано за счет каких библиотечных функций она работает.

1.1) Функции POSIX

Функция	**Назначение**
open	Открытие файла назначения.
close	Закрытие файловых дескрипторов после завершения копирования (уникальный id файла).
fstat	Получение размера исходного файла.
truncate	Обрезка файла назначения до реального размера.
posix_memalign	Выделение выровненной памяти.
free	Освобождение ранее выделенных буферов.

1.2) Функции libaio

Функция	**Назначение**
io_setup	Инициализация контекста асинхронного ввода-вывода.
io_destroy	Уничтожение контекста AIO и освобождение ресурсов ядра.
io_prep_pread	Подготовка структуры для асинхронного чтения.
io_prep_pwrite	Подготовка структуры для асинхронной записи.
io_submit	Отправка подготовленных операций в очередь ядра.
io_getevents	Ожидание и получение завершённых операций.

1.3) Структуры libaio

Структура	**Назначение**
iocb	Хранение параметров одной асинхронной операции.
io_event	Получение информации о завершённой операции: какой iocb завершился, сколько байт обработано, была ли ошибка.

2) Класс TestCopying

Класс TestCopying предназначен для автоматизированного тестирования производительности копирования файлов. Он создаёт набор тестовых файлов указанного размера, выполняет их копирование с помощью FileCopier, измеряет время выполнения и вычисляет средние показатели за несколько циклов. Класс TestCopying включает в себя три основных поля, которые описаны ниже в таблице.

Поле	**Тип**	**Описание**
numTests	int	Количество тестовых файлов.
fileSize	size_t	Размер каждого тестового файла.
fileCpy	FileCopier	Объект класса копировщика для тестирования.

3) Класс Interface

Класс Interface реализует консольный пользовательский интерфейс для удобного взаимодействия с программой. Он предоставляет меню для настройки параметров копирования, выполнения операций копирования, запуска тестов производительности и управления тестовыми файлами. Интерфейс программы представлен ниже (см Рис. 55).

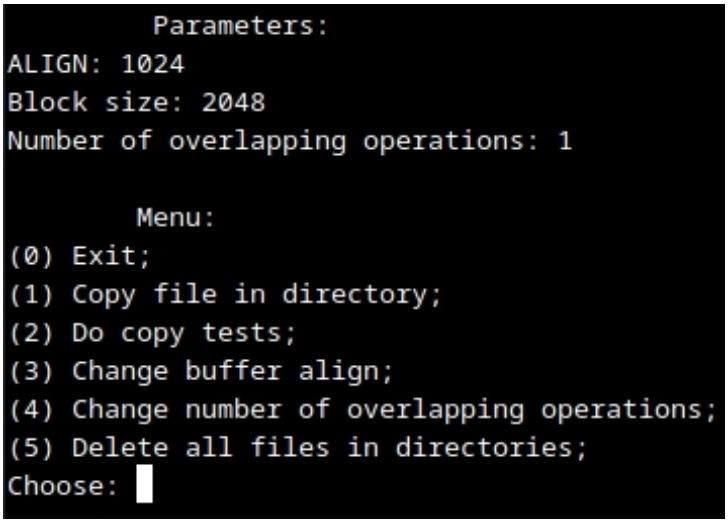


Рис. 55

4) Проверка зависимости производительности от размера блока буфера

Ниже показан интерфейс автотеста (см Рис. 56-58).

```
Number of files: 2
File size (KB): 10240
Runs: 1
Auto-test? (y/n): y
Auto-test block size (1) or overlap count (2): 1
Max block size (KB, step +2): 32
```

Рис. 56

```
Block size: 1 KB
Тестирование...
Среднее время выполнения (1 циклов): 7023 мс

Block size: 2 KB
Тестирование...
Среднее время выполнения (1 циклов): 3651 мс

Block size: 4 KB
Тестирование...
Среднее время выполнения (1 циклов): 1996 мс

Block size: 6 KB
Тестирование...
Среднее время выполнения (1 циклов): 1471 мс

Block size: 8 KB
Тестирование...
Среднее время выполнения (1 циклов): 1089 мс

Block size: 10 KB
Тестирование...
Среднее время выполнения (1 циклов): 908 мс

Block size: 12 KB
Тестирование...
Среднее время выполнения (1 циклов): 716 мс

Block size: 14 KB
Тестирование...
Среднее время выполнения (1 циклов): 657 мс

Block size: 16 KB
Тестирование...
Среднее время выполнения (1 циклов): 544 мс
```

Рис. 57

```
Block size: 18 KB
Тестирование...
Среднее время выполнения (1 циклов): 508 мс

Block size: 20 KB
Тестирование...
Среднее время выполнения (1 циклов): 460 мс

Block size: 22 KB
Тестирование...
Среднее время выполнения (1 циклов): 430 мс

Block size: 24 KB
Тестирование...
Среднее время выполнения (1 циклов): 374 мс

Block size: 26 KB
Тестирование...
Среднее время выполнения (1 циклов): 388 мс

Block size: 28 KB
Тестирование...
Среднее время выполнения (1 циклов): 360 мс

Block size: 30 KB
Тестирование...
Среднее время выполнения (1 циклов): 344 мс

Block size: 32 KB
Тестирование...
Среднее время выполнения (1 циклов): 301 мс

Tests done.
Press ENTER to continue...
```

Рис. 58

4.1)Анализ тестирования:

В первом тестировании я проверял зависимость скорости копирования от размера буфера, программа начала проверку с 1 КБ и дошла до 32 КБ. В автопроверке программа прогоняла файл весом 10 МБ. На графике видно, что от увеличения размера буфера уменьшается скорость копирования, так же видно, что есть разница от числового значения буфера (При буфере численно равном степени двойки производительность больше, чем при буфере не равном степени двойки). Ниже представлен график зависимости (см. Рис 59).

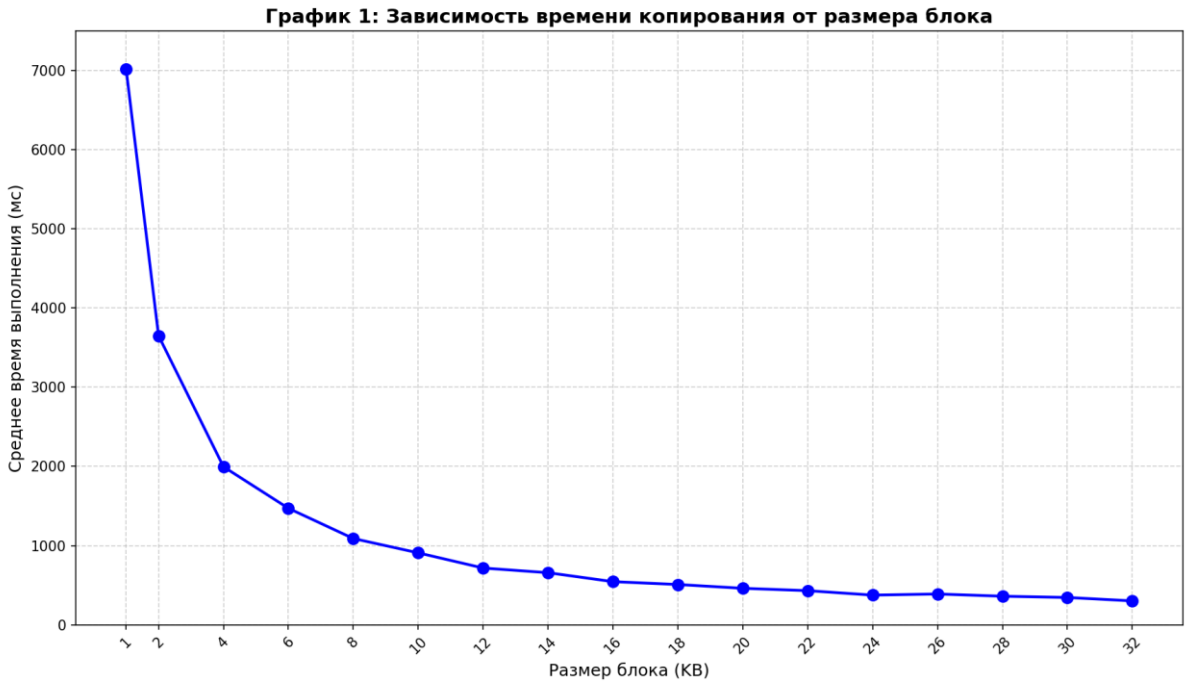


Рис. 59

5) Проверка зависимости производительности от количества операций

Ниже показан интерфейс автотеста (см Рис. 60-61).

```
        Parameters:
ALIGN: 1024
Block size: 1048576
Number of overlapping operations: 1
==========COPY TESTS==========
Number of files: 4
File size (KB): 16384
Runs: 2
Auto-test? (y/n): y
Auto-test block size (1) or overlap count (2): 2
Max overlap count (step +2): 12
Overlap count: 1
Тестирование...
Среднее время выполнения (2 циклов): 221.5 мс

Overlap count: 2
Тестирование...
Среднее время выполнения (2 циклов): 216.5 мс

Overlap count: 4
Тестирование...
Среднее время выполнения (2 циклов): 212 мс

Overlap count: 6
Тестирование...
Среднее время выполнения (2 циклов): 202 мс

Overlap count: 8
Тестирование...
Среднее время выполнения (2 циклов): 213.5 мс

Overlap count: 10
Тестирование...
Среднее время выполнения (2 циклов): 212 мс

Overlap count: 12
Тестирование...
Среднее время выполнения (2 циклов): 205.5 мс

Tests done.
Press ENTER to continue...
```

Рис. 60

```
        Parameters:
ALIGN: 1024
Block size: 2097152
Number of overlapping operations: 12
==========COPY TESTS==========
Number of files: 4
File size (KB): 16384
Runs: 2
Auto-test? (y/n): y
Auto-test block size (1) or overlap count (2): 2
Max overlap count (step +2): 12
Overlap count: 1
Тестирование...
Среднее время выполнения (2 циклов): 223.5 мс

Overlap count: 2
Тестирование...
Среднее время выполнения (2 циклов): 202.5 мс

Overlap count: 4
Тестирование...
Среднее время выполнения (2 циклов): 197.5 мс

Overlap count: 6
Тестирование...
Среднее время выполнения (2 циклов): 197.5 мс

Overlap count: 8
Тестирование...
Среднее время выполнения (2 циклов): 215.5 мс

Overlap count: 10
Тестирование...
Среднее время выполнения (2 циклов): 218 мс

Overlap count: 12
Тестирование...
Среднее время выполнения (2 циклов): 213.5 мс

Tests done.
Press ENTER to continue...
```

Рис. 61

5.1)Анализ тестирования:

Во втором тестировании была рассмотрена зависимость производительности от количества прерывающихся потоков. Для автотеста я взял два разных блока копирования (1 МБ и 2 МБ) и смотрел на производительность от 1 операции до 12 операций. На графике виден скачок производительности на 6 операциях, и виден линейный прирост с увеличением количества операций (см Рис. 62).

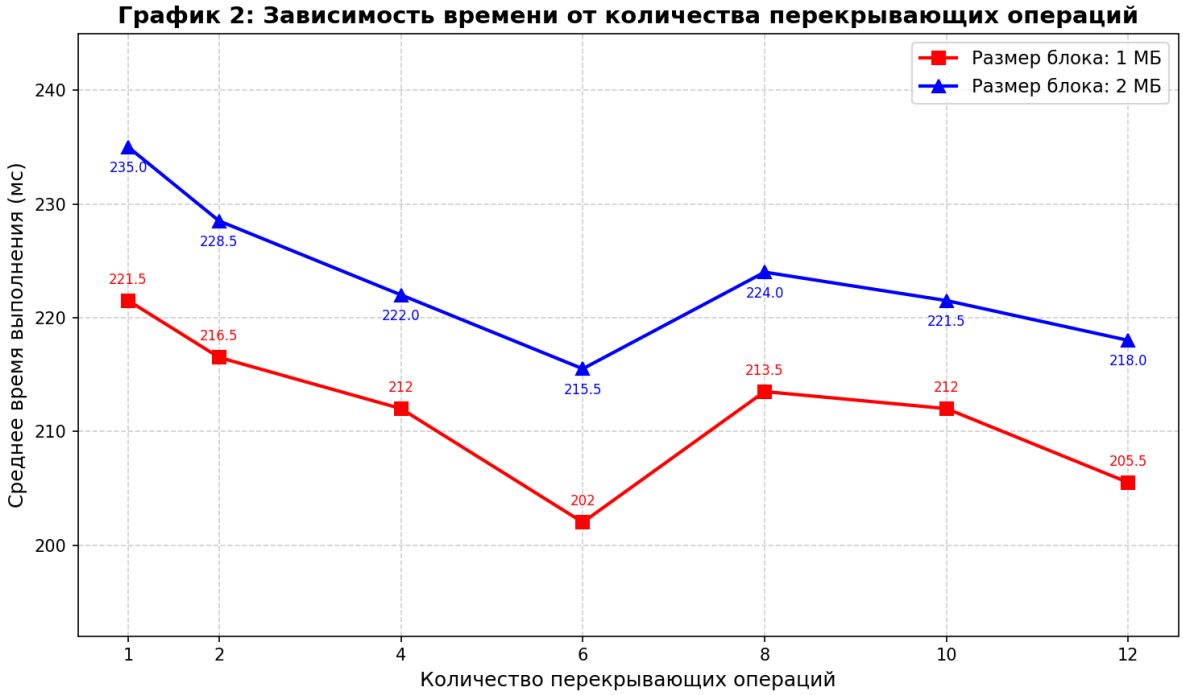


Рис. 62

Вывод

В первом задании было реализовано консольное приложение, позволяющее выполнять основные операции с дисками, каталогами и файлами. В ходе тестирования было установлено, что все используемые функции Win32 API работают корректно и предоставляют полноценный доступ к управлению файловой системой на низком уровне, однако требуют тщательной обработки ошибок и учёта прав доступа.

Во втором задании был проведён эксперимент по копированию файлов с использованием перекрывающегося (асинхронного) ввода-вывода. В ходе исследования зависимости скорости копирования от размера блока данных было выявлено, что увеличение размера блока с 1 до 20 КБ приводит к значительному снижению времени выполнения, что объясняется уменьшением количества системных вызовов и более эффективным использованием дискового кэша и шины ввода-вывода. Оптимальным размером блока в данном эксперименте можно считать

значение 20 КБ, однако уже при 8–10 КБ наблюдается существенное ускорение по сравнению с мелкими блоками. При исследовании влияния количества перекрывающих операций на производительность было установлено, что увеличение числа одновременно выполняемых операций ввода-вывода с 1 до 16 не даёт монотонного прироста скоростиЭто объясняется насыщением канала ввода-вывода и увеличением накладных расходов на синхронизацию при большом числе перекрывающихся операций.

Текст программы

1) fileManager.hpp

```
#include <iostream>
#include <windows.h>
#include <vector>
#include <string.h>
#include <iomanip>
#include "tools.hpp"

using namespace std;

void removeDirectory(const string& dirPath);

void createFile(const string& dirPath, const string& fileName, const string&
content = "");
void copyFile(const string& sourcePath, const string& destDir);
void moveFile(const string& sourcePath, const string& destPath);
void moveFileEx(const string& srcPath, const string& dstPath, DWORD flags);

void showFileAttributes(const string& filePath);
void setFileAttribute(const string& filePath);
void showFileInfo(const string& filePath);
void setFileTime(const string& filePath);
void showFileTimeInfo(const string& filePath);
```

2) fileManager.cpp

```
#include "fileManager.hpp"

void removeDirectory(const string& dirPath){
    string path =trimmed(dirPath);

    cout<<"Removing directory: "<<path<<endl;
```

```
    bool isDir;
    if(!checkPath(path, isDir)){
            cout<<"Error: Directory does not exist!"<<endl;
            return;
        }

    if(!isDir){
            cout<<"Error: Path is not a directory!"<<endl;
            return;
        }

    if(checkContain(path)){
        cout<<"Error: Directory is not empty! Please delete all files
first."<<endl;
        return;
        }

    if(RemoveDirectory(path.c_str())){
            cout<<"Directory successfully removed: "<<path<<endl;
            return;
    }else{
        DWORD error = GetLastError();
        if(error == ERROR_ACCESS_DENIED){
                cout<<"Error: Access denied!"<<endl;
        }else{
            cout<<"Error: failed to remove directory! Error code:
"<<error<<endl;
        }
        return;
    }
}

void createFile(const string& dirPath, const string& fileName, const string&
content){
    string fullPath = trimmed(dirPath)+ "\\" + fileName;

    cout<<"Creating file: "<<fullPath<<endl;

    HANDLE hFile = CreateFile(
        fullPath.c_str(),        //путь к файлу
        GENERIC_WRITE,           //доступ: запись
        0,                       //режим совместного доступа: исключительный
        NULL,                    //атрибуты безопасности: стандартные
        CREATE_ALWAYS,           //создаем новый/перезаписываем существующий
        FILE_ATTRIBUTE_NORMAL,   //обычный файл (без атрибутов)
        NULL                     //шаблон файла: не используется
    );
```

```
    if(hFile == INVALID_HANDLE_VALUE){
        cout<<"Error: failed to create file!"<<endl;
        return;
    }

    if(!content.empty()){
        DWORD bytesWritten;
        if(!WriteFile(hFile, content.c_str(), content.length(), &bytesWritten,
NULL)){
            cout<<"Error: failed to write to file!"<<endl;
            CloseHandle(hFile);
            return;
        }
        cout<<"Written "<<bytesWritten<<" bytes to file"<<endl;
    }
    //Закрываем дескриптор файла (освобождаем ресурс)
    CloseHandle(hFile);
    cout<<"File successfully created: "<<fullPath<<endl;
    return;
}

void copyFile(const string& sourcePath, const string& destDir){
    string copyFileName = getCopyFileName(trimmed(sourcePath));
    string destPath = trimmed(destDir)+ "\\" + copyFileName;

    bool destExists;
    bool destIsDir;
    destExists = checkPath(destPath, destIsDir);

    if(destExists && !destIsDir){
        cout<<"File already exists! Overwrite? (y/n): ";
        char choice;
        cin>>choice;
        if(tolower(choice)!= 'y'){
            cout<<"Copy cancelled."<<endl;
            return;
        }
    }

    if(CopyFile(trimmed(sourcePath).c_str(), destPath.c_str(), FALSE)){
        cout<<"File successfully copied!"<<endl;
        return;
    }else{
        cout<<"Error: failed to copy file!"<<endl;
        return;
    }
}
```

```
void moveFile(const string& sourcePath, const string& destPath){
    string src = trimmed(sourcePath);
    string dst = trimmed(destPath);
    string fileName = getFileName(src);

    bool isDir;
    bool destExists;
    bool destIsDir;

    if(checkPath(dst, isDir) && isDir){
        dst = dst + "\\" + fileName;
    }

    destExists = checkPath(dst, destIsDir);

    size_t pos = dst.find_last_of("\\/");
    string destDir = (pos == string::npos) ? "" : dst.substr(0, pos);

    bool dirExists = true;
    if(!destDir.empty()){
        if(!checkPath(destDir, dirExists) || !dirExists){
            cout << "Error: destination directory does not exist!" << endl;
            return;
        }
    }

    if(destExists && !destIsDir){
        cout<<"Destination file already exists!"<<endl;
        cout<<"Overwrite? (y/n): ";
        char choice;
        cin>>choice;

        if(tolower(choice) != 'y'){
            cout<<"Move cancelled."<<endl;
            return;
        }

        if(!DeleteFile(dst.c_str())){
            cout<<"Error: failed to delete existing file!"<<endl;
            return;
        }
    }
    else if(destExists && destIsDir){
        cout<<"Error: destination is a directory! Please specify a file
name."<<endl;
        return;
    }
```

```
    if(MoveFile(src.c_str(), dst.c_str())){
        cout<<"File successfully moved!"<<endl;
        cout<<"From: "<<src<<endl;
        cout<<"To: "<<dst<<endl;
        return;
    }else{
        DWORD error = GetLastError();

        if(error == ERROR_ACCESS_DENIED){
            cout<<"Error: Access denied!"<<endl;
        }
        else if(error == ERROR_NOT_SAME_DEVICE){
            cout<<"Error: Cannot move between different drives!"<<endl;
        }
        else{
            cout<<"Error: failed to move file! Error code: "<<error<<endl;
        }

        return;
    }
}

void moveFileEx(const string& srcPath, const string& dstPath, DWORD flags){
    BOOL result = MoveFileExA(srcPath.c_str(), dstPath.c_str(), flags);

    if (result)
    {
        cout << "Move successful\n";
        cout << "From: " << srcPath << endl;
        cout << "To:   " << dstPath << endl;
        return;
    }
    else
    {
        DWORD error = GetLastError();
        cout << "Move failed. Error code: " << error << endl;
        switch (error)
        {
            case ERROR_ACCESS_DENIED:
                cout << "Access denied. Check if:\n";
                cout << "  - Destination file is not read-only\n";
                cout << "  - Destination file is not open in another
program\n";
                cout << "  - You have write permissions\n";
                break;
            case ERROR_PATH_NOT_FOUND:cout << "Path not found. Check if
destination directory exists\n";break;
            case ERROR_FILE_NOT_FOUND:cout << "Source file not found\n";break;
```

```
            case ERROR_ALREADY_EXISTS:cout << "Destination file already
exists. Use REPLACE_EXISTING flag\n";break;
            case ERROR_NOT_SAME_DEVICE:cout << "Different drives. Use
MOVEFILE_COPY_ALLOWED flag\n";break;
            case ERROR_INVALID_PARAMETER:cout << "Invalid parameter. Check
paths and flags\n";break;
            default:cout << "Unhandled error\n";break;
        }

        return;
    }
}

void showFileAttributes(const string& filePath){
    DWORD attribute = GetFileAttributesA(filePath.c_str());
    if(attribute == INVALID_FILE_ATTRIBUTES){
        cout<<"Error: Could not get file attributes! Error code:
"<<GetLastError()<<endl;
        return;
    }

    cout<<"Attributes of file \""<<filePath<<"\":"<<endl;

    bool hasAttributes = false;

    if(attribute & FILE_ATTRIBUTE_DIRECTORY){
        cout<<" Directory"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_READONLY){
        cout<<" Read-only"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_HIDDEN){
        cout<<" Hidden"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_SYSTEM){
        cout<<" System"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_ARCHIVE){
        cout<<" Archive"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_TEMPORARY){
        cout<<" Temporary"<<endl;
        hasAttributes = true;
```

```
    }
    if(attribute & FILE_ATTRIBUTE_NORMAL){
        cout<<" Normal file"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_COMPRESSED){
        cout<<" Compressed"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_NOT_CONTENT_INDEXED){
        cout<<" Not content indexed"<<endl;
        hasAttributes = true;
    }
    if(attribute & FILE_ATTRIBUTE_ENCRYPTED){
        cout<<" Encrypted"<<endl;
        hasAttributes = true;
    }

    if(!hasAttributes){
        cout<<"  - No special attributes"<<endl;
    }
    cout<<"Attribute value (hex): 0x"<<hex<<attribute<<dec<<endl;
}

void setFileAttribute(const string& filePath){
    int indexAttribute;
    bool bAttribute;
    DWORD newAttribute;

    cout<<"Current attributes:"<<endl;
    showFileAttributes(filePath);

    cout<<"\nChoose attribute to set:"<<endl;
    cout<<"(1)  ARCHIVE"<<endl;
    cout<<"(2)  HIDDEN"<<endl;
    cout<<"(3)  NORMAL (clears all other attributes)"<<endl;
    cout<<"(4)  NOT CONTENT INDEXED"<<endl;
    cout<<"(5)  ENCRYPTED"<<endl;
    cout<<"(6)  READONLY"<<endl;
    cout<<"(7)  SYSTEM"<<endl;
    cout<<"(8)  TEMPORARY"<<endl;
    cout<<"(9)  COMPRESSED"<<endl;
    cout<<"Choose attribute: ";
    cin >> indexAttribute;

    switch(indexAttribute){
        case 1: newAttribute = FILE_ATTRIBUTE_ARCHIVE;break;
        case 2: newAttribute = FILE_ATTRIBUTE_HIDDEN;break;
```

```
        case 3: newAttribute = FILE_ATTRIBUTE_NORMAL;break;
        case 4: newAttribute = FILE_ATTRIBUTE_NOT_CONTENT_INDEXED;break;
        case 5: newAttribute = FILE_ATTRIBUTE_ENCRYPTED;break;
        case 6: newAttribute = FILE_ATTRIBUTE_READONLY;break;
        case 7: newAttribute = FILE_ATTRIBUTE_SYSTEM;break;
        case 8: newAttribute = FILE_ATTRIBUTE_TEMPORARY;break;
        case 9: newAttribute = FILE_ATTRIBUTE_COMPRESSED;break;
        default: cout<<"Invalid choice!"<<endl;return;
    }

    bAttribute = SetFileAttributesA(filePath.c_str(), newAttribute);
    if(bAttribute){
        cout<<"Attributes successfully changed!"<<endl;
        cout<<"New attributes:"<<endl;
        showFileAttributes(filePath);
    }
    else{
        cout<<"Error setting attributes! Error code: "<<GetLastError()<<endl;
        if(GetLastError() == ERROR_ACCESS_DENIED){
            cout<<"Access denied. Try running as administrator."<<endl;
        }
    }
}

void showFileInfo(const string& filePath){
    ULONGLONG fileSize;
    ULONGLONG fileId;

    BY_HANDLE_FILE_INFORMATION lpFileInformation;

    HANDLE hInfFile = CreateFile(
        filePath.c_str(),
        GENERIC_READ,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
        NULL,
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );

    if(hInfFile == INVALID_HANDLE_VALUE){
        cout<<"Error opening file! Error code: "<<GetLastError()<<endl;
        return;
    }

    if(GetFileInformationByHandle(hInfFile, &lpFileInformation)){
        cout<<"\nFile Information:"<<endl;
        cout<<"File: "<<filePath<<endl;
```

```
        //Attributes
        cout<<"Attributes: ";
        if(lpFileInformation.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY)
cout<<"Directory ";
        if(lpFileInformation.dwFileAttributes & FILE_ATTRIBUTE_READONLY)
cout<<"Read-only ";
        if(lpFileInformation.dwFileAttributes & FILE_ATTRIBUTE_HIDDEN)
cout<<"Hidden ";
        if(lpFileInformation.dwFileAttributes & FILE_ATTRIBUTE_SYSTEM)
cout<<"System ";
        if(lpFileInformation.dwFileAttributes & FILE_ATTRIBUTE_ARCHIVE)
cout<<"Archive ";
        if(lpFileInformation.dwFileAttributes & FILE_ATTRIBUTE_TEMPORARY)
cout<<"Temporary ";
        cout<<endl;

        //Time information
        printFileTimeForHandle(lpFileInformation.ftCreationTime, "Created");
        printFileTimeForHandle(lpFileInformation.ftLastAccessTime, "Last
access");
        printFileTimeForHandle(lpFileInformation.ftLastWriteTime, "Last
modified");

        //Size information
        fileSize =
(static_cast<ULONGLONG>(lpFileInformation.nFileSizeHigh)<<32) |
lpFileInformation.nFileSizeLow;
        cout<<"\nSize: "<<fileSize<<" bytes";
        if(fileSize > 1024) cout<<"
("<<fixed<<setprecision(2)<<(double)fileSize / 1024<<" KB)";
        if(fileSize > 1024 * 1024) cout<<"
("<<fixed<<setprecision(2)<<(double)fileSize / (1024 * 1024)<<" MB)";
        cout<<endl;

        //Volume information
        cout<<"Volume serial number:
0x"<<hex<<lpFileInformation.dwVolumeSerialNumber<<dec<<endl;

        //Link information
        cout<<"Number of hard links:
"<<lpFileInformation.nNumberOfLinks<<endl;
        if(lpFileInformation.nNumberOfLinks > 1){
            cout<<"  (This file has multiple hard links)"<<endl;
        }

        //File ID
```

```
        fileId =
(static_cast<ULONGLONG>(lpFileInformation.nFileIndexHigh)<<32) |
lpFileInformation.nFileIndexLow;
        cout<<"Unique file ID: 0x"<<hex<<fileId<<dec<<endl;

    }
    else{
        cout<<"Error getting file information! Error code:
"<<GetLastError()<<endl;
    }

    CloseHandle(hInfFile);
}

void showFileTimeInfo(const string& filePath){
    FILETIME createTime;
    FILETIME accessTime;
    FILETIME writeTime;

    HANDLE hFile = CreateFile(
        filePath.c_str(),    // Путь к файлу
        GENERIC_READ,        // Запрашиваем доступ только на чтение
        FILE_SHARE_READ,     // Разрешаем другим процессам читать файл
        NULL,                // Атрибуты безопасности (стандартные)
        OPEN_EXISTING,       // Открываем только существующий файл
        FILE_ATTRIBUTE_NORMAL, // Обычный файл
        NULL                 // Не используем шаблон файла
    );

    if(hFile == INVALID_HANDLE_VALUE){
        cout<<"Error opening file! Error code: "<<GetLastError()<<endl;
        return;
    }

    if(GetFileTime(hFile, &createTime, &accessTime, &writeTime)){
        cout<<"\nFile Time Information:"<<endl;
        cout<<"File: "<<filePath<<endl;
        printFileTimeForTime(createTime, "Created");
        printFileTimeForTime(accessTime, "Last accessed");
        printFileTimeForTime(writeTime, "Last modified");
    }
    else{
        cout<<"Failed to get file time! Error code: "<<GetLastError()<<endl;
    }

    CloseHandle(hFile);
}
```

```
void setFileTime(const string& filePath){
    int TimeID, year, month, day, hour, minute, second;
    DWORD flags, attrs;
    HANDLE hFile;
    FILETIME verifyCreate, verifyAccess, verifyWrite;

    attrs = GetFileAttributesA(filePath.c_str());
    if(attrs == INVALID_FILE_ATTRIBUTES){
        cout<<"Error: File does not exist! Error code:
"<<GetLastError()<<endl;
        return;
    }

    cout<<"\n=== Current File Times ==="<<endl;
    showFileTimeInfo(filePath);

    cout<<"\n=== Select time mode ==="<<endl;
    cout<<"(1) Set to current system time"<<endl;
    cout<<"(2) Set custom date and time"<<endl;
    cout<<"(3) Set creation time only"<<endl;
    cout<<"(4) Set last access time only"<<endl;
    cout<<"(5) Set last modified time only"<<endl;
    cout<<"Choose option: ";
    cin >> TimeID;

    flags = FILE_ATTRIBUTE_NORMAL;
    if(attrs & FILE_ATTRIBUTE_DIRECTORY){
        flags = FILE_FLAG_BACKUP_SEMANTICS;
        cout<<"Note: Target is a directory"<<endl;
    }

    hFile = CreateFile(
        filePath.c_str(),
        FILE_WRITE_ATTRIBUTES,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
        NULL,
        OPEN_EXISTING,
        flags,
        NULL
    );

    if(hFile == INVALID_HANDLE_VALUE){
        cout<<"Error opening file! Error code: "<<GetLastError()<<endl;
        return;
    }

    SYSTEMTIME st = {0};
    FILETIME newTime, newCreateTime, newAccessTime, newWriteTime;
```

```
    BOOL success = FALSE;

    switch (TimeID){
        case 1:
        {
            GetSystemTime(&st);
            SystemTimeToFileTime(&st, &newTime);
            cout<<"Setting all file times to current system time..."<<endl;
            success = SetFileTime(hFile, &newTime, &newTime, &newTime);
        }
        break;

        case 2:
        {
            cout<<"\nEnter custom date and time:"<<endl;
            cout<<"Year (1601-30827): ";
            cin >> year;
            cout<<"Month (1-12): ";
            cin >> month;
            cout<<"Day (1-31): ";
            cin >> day;
            cout<<"Hour (0-23): ";
            cin >> hour;
            cout<<"Minute (0-59): ";
            cin >> minute;
            cout<<"Second (0-59): ";
            cin >> second;

            st.wYear = year;
            st.wMonth = month;
            st.wDay = day;
            st.wHour = hour;
            st.wMinute = minute;
            st.wSecond = second;
            st.wMilliseconds = 0;

            if(st.wYear < 1601 || st.wMonth < 1 || st.wMonth > 12 ||
                st.wDay < 1 || st.wDay > 31 || st.wHour > 23 ||
                st.wMinute > 59 || st.wSecond > 59){
                cout<<"Error: Invalid date or time!"<<endl;
                CloseHandle(hFile);
                return;
            }

            if(!SystemTimeToFileTime(&st, &newTime)){
                cout<<"Error converting time! Error code:
"<<GetLastError()<<endl;
                CloseHandle(hFile);
```

```
                return;
            }

            cout<<"Setting all file times to custom date/time..."<<endl;
            success = SetFileTime(hFile, &newTime, &newTime, &newTime);
        }
        break;

        case 3:
        {
            cout<<"\nEnter custom creation date and time:"<<endl;
            cout<<"Year (1601-30827): ";
            cin >> year;
            cout<<"Month (1-12): ";
            cin >> month;
            cout<<"Day (1-31): ";
            cin >> day;
            cout<<"Hour (0-23): ";
            cin >> hour;
            cout<<"Minute (0-59): ";
            cin >> minute;
            cout<<"Second (0-59): ";
            cin >> second;

            st.wYear = year;
            st.wMonth = month;
            st.wDay = day;
            st.wHour = hour;
            st.wMinute = minute;
            st.wSecond = second;
            st.wMilliseconds = 0;

            if(st.wYear < 1601 || st.wMonth < 1 || st.wMonth > 12 ||
                st.wDay < 1 || st.wDay > 31 || st.wHour > 23 ||
                st.wMinute > 59 || st.wSecond > 59){
                cout<<"Error: Invalid date or time!"<<endl;
                CloseHandle(hFile);
                return;
            }

            if(!SystemTimeToFileTime(&st, &newCreateTime)){
                cout<<"Error converting time! Error code:
"<<GetLastError()<<endl;
                CloseHandle(hFile);
                return;
            }

            cout<<"Setting creation time only..."<<endl;
```

```
            success = SetFileTime(hFile, &newCreateTime, NULL, NULL);
        }
        break;

        case 4:
        {
            cout<<"\nEnter custom last access date and time:"<<endl;
            cout<<"Year (1601-30827): ";
            cin >> year;
            cout<<"Month (1-12): ";
            cin >> month;
            cout<<"Day (1-31): ";
            cin >> day;
            cout<<"Hour (0-23): ";
            cin >> hour;
            cout<<"Minute (0-59): ";
            cin >> minute;
            cout<<"Second (0-59): ";
            cin >> second;

            st.wYear = year;
            st.wMonth = month;
            st.wDay = day;
            st.wHour = hour;
            st.wMinute = minute;
            st.wSecond = second;
            st.wMilliseconds = 0;

            if(st.wYear < 1601 || st.wMonth < 1 || st.wMonth > 12 ||
                st.wDay < 1 || st.wDay > 31 || st.wHour > 23 ||
                st.wMinute > 59 || st.wSecond > 59){
                cout<<"Error: Invalid date or time!"<<endl;
                CloseHandle(hFile);
                return;
            }

            if(!SystemTimeToFileTime(&st, &newAccessTime)){
                cout<<"Error converting time! Error code:
"<<GetLastError()<<endl;
                CloseHandle(hFile);
                return;
            }

            cout<<"Setting last access time only..."<<endl;
            success = SetFileTime(hFile, NULL, &newAccessTime, NULL);
        }
        break;
```

```
        case 5:
        {
            cout<<"\nEnter custom last modified date and time:"<<endl;
            cout<<"Year (1601-30827): ";
            cin >> year;
            cout<<"Month (1-12): ";
            cin >> month;
            cout<<"Day (1-31): ";
            cin >> day;
            cout<<"Hour (0-23): ";
            cin >> hour;
            cout<<"Minute (0-59): ";
            cin >> minute;
            cout<<"Second (0-59): ";
            cin >> second;

            st.wYear = year;
            st.wMonth = month;
            st.wDay = day;
            st.wHour = hour;
            st.wMinute = minute;
            st.wSecond = second;
            st.wMilliseconds = 0;

            if(st.wYear < 1601 || st.wMonth < 1 || st.wMonth > 12 ||
                st.wDay < 1 || st.wDay > 31 || st.wHour > 23 ||
                st.wMinute > 59 || st.wSecond > 59){
                cout<<"Error: Invalid date or time!"<<endl;
                CloseHandle(hFile);
                return;
            }

            if(!SystemTimeToFileTime(&st, &newWriteTime)){
                cout<<"Error converting time! Error code:
"<<GetLastError()<<endl;
                CloseHandle(hFile);
                return;
            }

            cout<<"Setting last modified time only..."<<endl;
            success = SetFileTime(hFile, NULL, NULL, &newWriteTime);
        }
        break;

        default:
            cout<<"Error: Invalid choice (1-5)!"<<endl;
            CloseHandle(hFile);
            return;
```

```
    }

    if(success){
        cout<<"\nFile times successfully updated!"<<endl;
        cout<<"\n=== New File Times ==="<<endl;

        if(GetFileTime(hFile, &verifyCreate, &verifyAccess, &verifyWrite)){
           printFileTimeForTime(verifyCreate, "Created");
           printFileTimeForTime(verifyAccess, "Last accessed");
           printFileTimeForTime(verifyWrite, "Last modified");
        }
    }else{
        cout<<"Error setting file times! Error code: "<<GetLastError()<<endl;
        if(GetLastError() == ERROR_ACCESS_DENIED){
            cout<<"Access denied. Try running as administrator."<<endl;
        }
    }
    CloseHandle(hFile);
}
```

3) driversManager.hpp

```
#include <iostream>
#include <windows.h>
#include <vector>
#include <string.h>
#include <iomanip>
#include "tools.hpp"

using namespace std;

#define MAX_SIZE 250

void showLogicalDrives(vector<string>& listofdrivers); //Вывод списка дисков
(через битовую маску)
void showLogicalDrivesSTR(vector<string>& listofdrivers); //Вывод списка
дисков (через список строк)
void showDriveType(const vector<string>& listofdrivers); //Вывод типа диска
void showDriverInformation(const vector<string>& listofdrivers);//Вывод
детальной  информации о томе
void showDriverFreeSpace(const vector<string>& listofdrivers); //Получает
информацию о свободном месте на диске
```

4) driversManager.cpp

```
#include "driversManager.hpp"

//Вывод дисков в системе
void showLogicalDrives(vector<string>& listofdrivers){
    int count = 1;
    listofdrivers.clear();
```

```
    DWORD drives = GetLogicalDrives();          //Получение битовой маски
дисков
    cout<<"Your drives: "<<endl;
    //Проверка битов и формирование пути для существующих дисков
    for(char d = 'A'; d <= 'Z'; d++){
        if(drives & 1){                         //Проверка младшего бита
(текущий диск)
            string disk = string(1, d) + ":\\";
            cout<<count<<". "<<disk<<endl;
            listofdrivers.push_back(disk);
            count++;
        }
        drives >>= 1;                           //Сдвиг вправо для проверки
следующего бита
    }
}

//Вывод списка дисков (через список строк)
void showLogicalDrivesSTR(vector<string>& listofdrivers){
    char lpBuffer[MAX_SIZE];                    //Буфер для приема строк от
API
    int count = 1;
    listofdrivers.clear();

    DWORD result = GetLogicalDriveStringsA(MAX_SIZE, lpBuffer);  //Заполняет
буфер: "C:\0D:\0\0". Возвращает кол-во символов или 0
    //Парсинг буфера, перебор строк до двойного null, и добавление пути в
список listofdrivers
    if((result > 0) && (result <= MAX_SIZE)){
        cout<<"Your drives: "<<endl;
        char* drive = lpBuffer;                 //Указатель на начало буфера
        while(*drive){
            cout<<count<<". "<<drive<<endl;
            listofdrivers.push_back(string(drive));
            drive += strlen(drive) + 1;
            count++;
        }
    }else{
        cout<<"Error getting drive list!"<<endl;
    }
}

//Вывод типа диска
void showDriveType(const vector<string>& listofdrivers){
    if(listofdrivers.empty()){
        cout<<"Please check your drives first (Option 1-2 in menu)"<<endl;
        return;
    }
```

```
    int id;
    int number;
    cout<<"Please select a disk number: ";
    cin >> number;
    id = number - 1;

    if(id < 0 || id >= (int)listofdrivers.size()){
        cout<<"Invalid disk number!"<<endl;
        return;
    }

    UINT type = GetDriveTypeA(listofdrivers[id].c_str());
    switch(type){
        case 0:cout<<"DRIVE_UNKNOWN"<<endl;break;
        case 1:cout<<"DRIVE_NO_ROOT_DIR "<<endl;break;
        case 2:cout<<"DRIVE_REMOVABLE"<<endl;break;
        case 3:cout<<"DRIVE_FIXED"<<endl;break;
        case 4:cout<<"DRIVE_REMOTE"<<endl;break;
        case 5:cout<<"DRIVE_CDROM"<<endl;break;
        case 6:cout<<"DRIVE_RAMDISK"<<endl;break;
        default:cout<<"Unknown drive type: "<<type<<endl; break;
    }
}

//Вывод детальной  информации о томе
void showDriverInformation(const vector<string>& listofdrivers){
    if(listofdrivers.empty()){
        cout<<"Please check your drives first (Option 1-2 in menu)"<<endl;
        return;
    }

    //Буферы для приема данных от API
    CHAR VolumeNameBuffer[MAX_SIZE];      //буфер для имени тома
    CHAR FileSystemNameBuffer[MAX_SIZE];  //буфер для файловой системы (NTFS,
FAT32)
    DWORD componentlenght;                //максимальная длина компоненты
имени файла
    DWORD flags;                          //флаги возможностей файловой
системы
    DWORD serial;                         //серийный номер тома

    int id;
    cout<<"Please select a disk number: ";
    cin >> id;

    int index = id - 1;
```

```
    if(index < 0 || index >= (int)listofdrivers.size()){
        cout<<"Invalid disk number!"<<endl;
        return;
    }

    BOOL info = GetVolumeInformationA(
        listofdrivers[index].c_str(),   //путь к диску
        VolumeNameBuffer,
        MAX_SIZE,                       //размер буфера
        &serial,
        &componentlenght,
        &flags,
        FileSystemNameBuffer,
        MAX_SIZE
    );

    if(info){
        cout<<"Disk name: "<<VolumeNameBuffer<<endl;
        cout<<"Serial number: "<<serial<<endl;
        cout<<"File system: "<<FileSystemNameBuffer<<endl;
        cout<<"Max length of file name component: "<<componentlenght<<endl;
        cout<<"File system info: "<<endl;

        if(flags & FILE_CASE_SENSITIVE_SEARCH) cout<<"  (+) Case-sensitive
search"<<endl; //Файловая система чувствительна к регистру
        if(flags & FILE_CASE_PRESERVED_NAMES) cout<<"  (+) Preserves case of
filenames"<<endl; //Сохраняет регистр имен файлов
        if(flags & FILE_UNICODE_ON_DISK) cout<<"  (+) Unicode
filenames"<<endl; //Поддерживает Unicode в именах
        if(flags & FILE_FILE_COMPRESSION) cout<<"  (+) File-based
compression"<<endl; //Поддерживает сжатие файлов
        if(flags & FILE_VOLUME_QUOTAS) cout<<"  (+) Disk quotas"<<endl;
//Поддерживает квоты диска
        if(flags & FILE_SUPPORTS_SPARSE_FILES) cout<<"  (+) Sparse
files"<<endl; //Поддерживает разреженные файлы
        if(flags & FILE_VOLUME_IS_COMPRESSED) cout<<"  (+) Volume is
compressed"<<endl; //Весь том сжат
        if(flags & FILE_SUPPORTS_ENCRYPTION) cout<<"  (+) Encrypted File
System (EFS)"<<endl; //Поддерживает шифрование (EFS)
        if(flags & FILE_NAMED_STREAMS) cout<<"  (+) NTFS"<<endl;
//Поддерживает именованные потоки (NTFS)
        if(flags & FILE_READ_ONLY_VOLUME) cout<<"  (+) Volume is read-
only"<<endl; //Том только для чтения
        if(flags & FILE_SUPPORTS_TRANSACTIONS) cout<<"  (+)
Transactions"<<endl; //Поддерживает транзакции (TxF)
        if(flags & FILE_SUPPORTS_HARD_LINKS) cout<<"  (+) Hard links"<<endl;
//Поддерживает жесткие ссылки
    }
```

```
    else{
        cout<<"Error getting volume information! Error code:
"<<GetLastError()<<endl;
    }
}

//Получает информацию о свободном месте на диске
void showDriverFreeSpace(const vector<string>& listofdrivers){
    if(listofdrivers.empty()){
        cout<<"Please check your drives first (Option 1-2 in menu)"<<endl;
        return;
    }

    DWORD SectorsPerCluster; // количество секторов в кластере
    DWORD BytesPerSector; // количество байт в секторе
    DWORD NumberOfFreeClusters; // количество свободных кластеров
    DWORD TotalNumberOfClusters; // общее количество кластеров

    int id;
    int number;
    cout<<"Please select a disk number: ";
    cin >> number;

    id = number - 1;

    if(id < 0 || id >= (int)listofdrivers.size()){
        cout<<"Invalid disk number!"<<endl;
        return;
    }

    BOOL space = GetDiskFreeSpaceA(
        listofdrivers[id].c_str(),
        &SectorsPerCluster,
        &BytesPerSector,
        &NumberOfFreeClusters,
        &TotalNumberOfClusters
    );

    if(space){
        __int64 totalBytes = (__int64)TotalNumberOfClusters *
SectorsPerCluster * BytesPerSector;
        __int64 freeBytes = (__int64)NumberOfFreeClusters * SectorsPerCluster
* BytesPerSector;

        cout<<"(1) Sectors per cluster: "<<SectorsPerCluster<<endl;
        cout<<"(2) Bytes per sector: "<<BytesPerSector<<endl;
        cout<<"(3) Number of free clusters: "<<NumberOfFreeClusters<<endl;
        cout<<"(4) Total clusters: "<<TotalNumberOfClusters<<endl;
```

```
        cout<<"(5) Total disk space: "<<totalBytes / (1024 * 1024)<<"
MB"<<endl;
        cout<<"(6) Free disk space: "<<freeBytes / (1024 * 1024)<<" MB"<<endl;
    }
    else{
        cout<<"Error getting disk free space! Error code:
"<<GetLastError()<<endl;
    }
}
```

6)tools.hpp

```
#include <iostream>
#include <windows.h>
#include <vector>
#include <string.h>
#include <iomanip>

using namespace std;

string trimmed(const string& str);
void printFileTimeForTime(const FILETIME& ftime,const string& labe);
void printFileTimeForHandle(const FILETIME& ft, const char* label);
bool checkPath(const string& path, bool& isDirectory);
string getFileName(const string& path);
string getCopyFileName(const string& originalPath);
bool checkContain(const string& fullPath);
```

7)tools.cpp

```
#include "tools.hpp"

string trimmed(const string& str){
    size_t first = str.find_first_not_of(" \t\n\r");
    if(first == string::npos)return "";
    size_t last = str.find_last_not_of(" \t\n\r");
    return str.substr(first, last - first + 1);
}


void printFileTimeForTime(const FILETIME& ftime,const string& labe){
        if(ftime.dwHighDateTime == 0 && ftime.dwLowDateTime ==0){
            cout<<labe<<": Not avalible"<<endl;
            return;
        }
        SYSTEMTIME stime;
        FileTimeToSystemTime(&ftime,&stime);

        cout<<labe<<":
"<<std::setfill('0')<<std::setw(2)<<stime.wDay<<"."<<std::setw(2)<<stime.wMont
```

```
h<<"."<<std::setw(4)<<stime.wYear<<"
"<<std::setw(2)<<stime.wHour<<":"<<std::setw(2)<<stime.wMinute<<":"<<std::setw
(2)<<stime.wSecond<<std::setfill(' ')<<endl;
    }

void printFileTimeForHandle(const FILETIME& ft, const char* label){
    SYSTEMTIME st;
    FileTimeToSystemTime(&ft, &st);
    std::cout<<label<<": "<<std::setfill('0')<<
st.wDay<<"."<<std::setw(2)<<st.wMonth<<"."<<st.wYear<<" "<<
std::setw(2)<<st.wHour<<":"<<std::setw(2)<<st.wMinute<<":"<<std::setw(2)<<st.w
Second<<std::endl;
}


bool checkPath(const string& path, bool& isDirectory){
    DWORD attrs = GetFileAttributes(path.c_str());
    if(attrs == INVALID_FILE_ATTRIBUTES){
        return false;
    }
    isDirectory = (attrs & FILE_ATTRIBUTE_DIRECTORY)!= 0;
    return true;
}

string getFileName(const string& path){
    size_t pos = path.find_last_of("\\");
    if(pos != string::npos){
        return path.substr(pos + 1);
    }
    return path;
}

string getCopyFileName(const string& originalPath){
    string fileName = getFileName(originalPath);
    size_t dotPos = fileName.find_last_of(".");

    if(dotPos != string::npos){
        string nameWithoutExt = fileName.substr(0, dotPos);
        string ext = fileName.substr(dotPos);
        return nameWithoutExt + "_copy" + ext;
    }else{
        return fileName + "_copy";
    }
}

bool checkContain(const string& fullPath){
    string searchPath = fullPath + "\\*";
    WIN32_FIND_DATA findData;
```

```
    HANDLE hFind = FindFirstFile(searchPath.c_str(), &findData);

   if(hFind == INVALID_HANDLE_VALUE){
        return false;
    }

    do{
       if(strcmp(findData.cFileName, ".") != 0 &&
            strcmp(findData.cFileName, "..") != 0){
            FindClose(hFind);
            return true;
        }
    } while (FindNextFile(hFind, &findData));

    FindClose(hFind);
    return false;
}
```

8)main.cpp

```
#include "tools.hpp"
#include "fileManager.hpp"
#include "driversManager.hpp"

int main()
{
    int flag = 1;
    int menu;
    vector<string> listofdrivers;

    while(flag){
        cout<<"\n           Menu:\n";
        cout<<"(0)  Exit;"<<endl;
        cout<<"(1)  Output list of disks;"<<endl;
        cout<<"(2)  Output list of disks;"<<endl;
        cout<<"(3)  Driver Type;"<<endl;
        cout<<"(4)  Volume Information;"<<endl;
        cout<<"(5)  Disk free space;"<<endl;
        cout<<"(6)  Create directory;"<<endl;
        cout<<"(7)  Remove directory;"<<endl;
        cout<<"(8)  Create file in directory;"<<endl;
        cout<<"(9)  Copy file;"<<endl;
        cout<<"(10) Move file;"<<endl;
        cout<<"(11) Move file (extended);"<<endl;
        cout<<"(12) File attributes;"<<endl;
        cout<<"(13) Set attributes;"<<endl;
        cout<<"(14) Get file info;"<<endl;
        cout<<"(15) Get file time;"<<endl;
        cout<<"(16) Set file time;"<<endl;
        cout<<"\nChoose function: ";
```

```
        cin>>menu;
        cout<<"\n";

        switch(menu){
            case 1:
                showLogicalDrives(listofdrivers);
                break;
            case 2:
                showLogicalDrivesSTR(listofdrivers);
                break;
            case 3:
                showDriveType(listofdrivers);
                break;
            case 4:
                showDriverInformation(listofdrivers);
                break;
            case 5:
                showDriverFreeSpace(listofdrivers);
                break;
            case 6:
            {
                string dirPath;
                string dirName;
                bool isDir;
                cin.ignore();
                cout<<"Enter directory Path: ";
                getline(cin, dirPath);
                //Проверка на существование родительской директории
                if(!checkPath(trimmed(dirPath), isDir) || !isDir){
                    cout<<"Error: Parent directory does not exist:
"<<dirPath<<endl;
                    break;
                }
                cout<<"Enter directory name: ";
                getline(cin, dirName);
                string fullPath = trimmed(dirPath) + "\\" + trimmed(dirName);
                //Защита от повторение директорий
                if(checkPath(fullPath, isDir)){
                    if(isDir){
                        cout<<"Warning: Directory already exists!"<<endl;
                        break;
                    }
                }
                //Вызов API с нулевыми критериями безопастности
                if(CreateDirectory(fullPath.c_str(), NULL)){
                    cout<<"Directory successfully created: "<<fullPath<<endl;
                    break;
                }else{
```

```
                    //Обработка исключений
                    DWORD error = GetLastError();
                    cout<<"Error: failed to create directory! Error code:
"<<error<<endl;
                    break;
                }
                break;
            }
            case 7:
            {
                string dirPath;
                cin.ignore();
                cout<<"Enter directory Path: ";
                getline(cin, dirPath);
                removeDirectory(dirPath);
                break;
            }
            case 8:
            {
                string dirPath;
                string fileName;
                string content;
                cin.ignore();
                cout<<"Enter directory path: ";
                getline(cin, dirPath);
                bool isDir;
                //Проверка существования родительской директории
                if(!checkPath(trimmed(dirPath), isDir)|| !isDir){
                    cout<<"Error: directory does not exist:
"<<trimmed(dirPath)<<endl;
                    break;
                }
                cout<<"Enter file name: ";
                getline(cin, fileName);
                cout<<"Enter file content (Enter for empty): ";
                getline(cin, content);
                createFile(dirPath, fileName, content);
                break;
            }
            case 9:
            {
                string sourcePath;
                string destDir;
                cin.ignore();
                cout<<"Enter source file path: ";
                getline(cin, sourcePath);
                bool isDir;
                if(!checkPath(trimmed(sourcePath), isDir)){
```

```
                    cout<<"Error: directory "<<trimmed(sourcePath)<<" is
incorrect"<< endl;
                    break;
                }
                cout<<"Enter destination directory: ";
                getline(cin, destDir);
                if(!checkPath(trimmed(destDir), isDir)){
                    cout<<"Error: directory "<<trimmed(destDir)<<" is
incorrect"<< endl;
                    break;
                }
                copyFile(sourcePath, destDir);
                break;
            }
            case 10:
            {
                string sourcePath;
                string destPath;
                bool isDir;
                cin.ignore();
                cout<<"Enter source file path: ";
                getline(cin, sourcePath);
                if(!checkPath(trimmed(sourcePath), isDir)){
                    cout<<"Error: source file does not exist!"<<endl;
                    break;
                }
                if(isDir){
                    cout<<"Error: source is a directory!"<<endl;
                    break;
                }
                cout<<"Enter destination: ";
                getline(cin, destPath);
                moveFile(sourcePath, destPath);
                break;
            }
            case 11:
            {
                string sourcePath;
                string destPath;
                int flagChoice;

                cin.ignore(); // очистка буфера

                cout << "Enter source file path: ";
                getline(cin, sourcePath);

                bool isDir;
```

```
                if (!checkPath(trimmed(sourcePath), isDir)) {
                    cout << "Error: source file does not exist!" << endl;
                    break;
                }

                if (isDir) {
                    cout << "Error: source is a directory!" << endl;
                    break;
                }

                cout << "Enter destination: ";
                getline(cin, destPath);

                bool destIsDir;
                string finalDestPath = destPath;

                if (checkPath(destPath, destIsDir) && destIsDir) {
                    size_t lastSlash = sourcePath.find_last_of("\\/");
                    string fileName = (lastSlash != string::npos) ?
                                    sourcePath.substr(lastSlash + 1) :
sourcePath;
                    if (destPath.back() != '\\' && destPath.back() != '/') {
                        finalDestPath += '\\';
                    }
                    finalDestPath += fileName;
                }

                cout << "\nSelect MoveFileEx flags:" << endl;
                cout << "(1) Normal move;" << endl;
                cout << "(2) Replace existing file;" << endl;
                cout << "(3) Allow copy between different drives;" << endl;
                cout << "\nChoose flag option: ";
                cin >> flagChoice;

                DWORD flags = 0;
                switch (flagChoice) {
                    case 1:flags = MOVEFILE_COPY_ALLOWED; break;
                    case 2:flags = MOVEFILE_REPLACE_EXISTING |
MOVEFILE_COPY_ALLOWED;break;
                    case 3:flags = MOVEFILE_COPY_ALLOWED; break;
                    default:
                        cout << "Invalid option. Using default
(COPY_ALLOWED).\n";
                        flags = MOVEFILE_COPY_ALLOWED;
                        break;
                }

                moveFileEx(sourcePath, finalDestPath, flags);
```

```
            break;
        }
        case 12:
        {
            string filePath;
            cin.ignore();
            cout<<"Enter file path: ";
            getline(cin, filePath);
            showFileAttributes(filePath);
            break;
        }
        case 13:
        {
            string filePath;
            cin.ignore();
            cout<<"Enter file path: ";
            getline(cin, filePath);
            setFileAttribute(filePath);
            break;
        }
        case 14:
        {
            string filePath;
            cin.ignore();
            cout<<"Enter file path: ";
            getline(cin, filePath);
            showFileInfo(filePath);
            break;
        }
        case 15:
        {
            string filePath;
            cin.ignore();
            cout<<"Enter file path: ";
            getline(cin, filePath);
            showFileTimeInfo(filePath);
            break;
        }
        case 16:
        {
            string filePath;
            cin.ignore();
            cout<<"Enter file path: ";
            getline(cin, filePath);
            setFileTime(filePath);
            break;
        }
```

```
            case 0:
                flag = 0;
                break;
            default:
                cout<<"Invalid choice! Please select 0-16."<<endl;
                break;
        }
    }
    return 0;
}
```

9) FileCopier.hpp

```
#pragma once
#include <string>
#include <libaio.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <iostream>
#include <filesystem>
#include <vector>

class FileCopier
{
private:
    size_t ALIGN; //выравнивание буфера в байтах
    size_t blockSize; //размер блока копирования (кратен ALIGN)
    int overlapCount; //число перекрывающих операций ввода и вывода
public:
    FileCopier();
    FileCopier(const size_t _ALIGN, const int blockSizeMultiplier);
    ~FileCopier();
    bool copyFile(const std::string& srcPath, const std::string& destDir);

    FileCopier& operator=(const FileCopier& _fileCpy);
    void setAlign(const size_t _ALIGN, const int blockSizeMultiplier); //ALIGN
в килобайтах
    void setOverlapCount(const int _overlapCount);

    size_t getAlign();
    size_t getBlockSize();
    int getOverlapCount();
};
```

10) FileCopier.cpp

```
#include "FileCopier.hpp"

FileCopier::FileCopier(){}
```

```
FileCopier::FileCopier(const size_t _ALIGN, const int
blockSizeMultiplier):ALIGN(_ALIGN*1024){
    blockSize = ALIGN*blockSizeMultiplier;
    overlapCount=1;
}

FileCopier::~FileCopier(){}

void FileCopier::setAlign(const size_t _ALIGN, const int blockSizeMultiplier){
    ALIGN=_ALIGN*1024;
    blockSize=ALIGN*blockSizeMultiplier;
}

void FileCopier::setOverlapCount(const int _overlapCount){
    overlapCount=_overlapCount;
}

size_t FileCopier::getAlign(){return ALIGN;}
size_t FileCopier::getBlockSize(){return blockSize;}
int FileCopier::getOverlapCount(){return overlapCount;}

FileCopier& FileCopier::operator=(const FileCopier& _fileCpy){
    if(this==&_fileCpy){
        return *this;
    }
    ALIGN=_fileCpy.ALIGN;
    blockSize=_fileCpy.blockSize;
    overlapCount=_fileCpy.overlapCount;
    return *this;
}

bool FileCopier::copyFile(const std::string& srcPath, const std::string&
destDir){
    //извлекаем имя файла из srcPath и склеиваем с destDir
    std::string filename = std::filesystem::path(srcPath).filename();
    std::string destPath  = std::filesystem::path(destDir) / filename;

    //открытие файла
    //O_RDONLY - только чтение
    //O_DIRECT - обход page cache: данные идут напрямую диск - буфер
    int srcFd = open(srcPath.c_str(), O_RDONLY | O_DIRECT);
    if(srcFd<0){
        std::cout << "Не удалось открыть исходный файл: " + srcPath;
        return false;
    }

    //O_WRONLY - только запись
    //O_CREAT - создать файл если не существует
```

```
    //O_TRUNC - обнулить файл если существует
    //O_DIRECT - тот же смысл, что и для источника
    //0644 устанавливает права для чтения и записи (6 - владелец и читать и
писать, остальные 4 - только читать)
    int dstFd = open(destPath.c_str(), O_WRONLY | O_CREAT | O_TRUNC |
O_DIRECT, 0644);
    if(dstFd<0){
        std::cout << ("Не удалось создать файл назначения: " + destPath);
        close(srcFd);
        return false;
    }

    //Узнаём размер исходного файла
    struct stat fileStat;
    fstat(srcFd, &fileStat);
    off_t fileSize = fileStat.st_size;

    //O_DIRECT требует чтобы размер блока был кратен ALIGN
    //округляем fileSize вверх до кратного ALIGN для последнего блока
    //это нужно чтобы последний неполный блок тоже читался корректно
    off_t alignedSize = ((fileSize+ALIGN-1) / ALIGN) * ALIGN;

    //Выделение массивов буферов и iocb по n штук
    //каждый слот i имеет свой readBuf[i], writeBuf[i], readCb[i], writeCb[i]
    //это позволяет n операциям чтения и n операциям записи работать
параллельно
    std::vector<void*> readBufs(overlapCount, nullptr);
    std::vector<void*> writeBufs(overlapCount, nullptr);
    for(int i=0; i<overlapCount; i++){
        posix_memalign(&readBufs[i],  ALIGN, blockSize);
        posix_memalign(&writeBufs[i], ALIGN, blockSize);
    }

    //iocb — структура описания одной асинхронной операции (I/O Control Block)
    //по n штук для чтения и записи - каждый слот независим
    std::vector<iocb> readCbs(overlapCount);
    std::vector<iocb> writeCbs(overlapCount);

    //bytesRead[i] - сколько байт прочитал слот i (нужно при завершении
записи)
    std::vector<size_t> bytesReadArr(overlapCount, 0);

    //Инициализация контекста AIO
    //io_context_t - дескриптор очереди асинхронных операций
    //io_setup - максимум 2*n операций одновременно
    io_context_t ctx = {};
    io_setup(2*overlapCount, &ctx);
```

```
    off_t readOffset  = 0;  //текущая позиция чтения в исходном файле
    off_t writeOffset = 0;  //текущая позиция записи в файле назначения
    int inFlight = 0;  //счётчик операций находящихся сейчас в очереди ядра

    struct iocb* cbs[2];
    int rc;

    //Запускаем первые n операций чтения чтобы заполнить очередь
    for(int i=0; i<overlapCount && readOffset<alignedSize; i++){
        size_t toRead = std::min((off_t)blockSize, alignedSize - readOffset);

        //заполняет структуру iocb для операции чтения count байт из fd
начиная со смещения offset в буфер buf
        io_prep_pread(&readCbs[i], srcFd, readBufs[i], toRead, readOffset);

        //сохраняем индекс слота в поле data, чтобы в цикле понять, какой
именно слот завершился когда придёт событие
        readCbs[i].data = (void*)(intptr_t)i;

        cbs[0] = &readCbs[i];

        //отправляет n операций в очередь ядра, после этого вызова операция
выполняется асинхронно
        rc = io_submit(ctx, 1, cbs);
        if(rc<0){
            break;
        }
        readOffset += toRead;
        inFlight++;
    }

    //Основной цикл перекрывающегося копирования
    while(inFlight>0){
        struct io_event events[2*overlapCount];

        //минимум 1 событие, максимум inFlight событий
        //поток спит до прихода completion event
        int nEvents = io_getevents(ctx, 1, inFlight, events, nullptr);
        if(nEvents<0){
            break;
        }

        inFlight -= nEvents; //уменьшаем счётчик на число завершившихся
операций

        for(int i=0; i<nEvents; i++){
            struct iocb* completed = events[i].obj; //какая именно операция
завершилась
```

```
            //получаем индекс слота из поля data которое мы сами туда записали
            int slot = (int)(intptr_t)completed->data;

            //число обработанных байт
            if(events[i].res < 0){
                continue;
            }

            if(completed == &readCbs[slot]){
                //Чтение слота завершено
                //реально прочитанных байт — не больше чем размер файла
                bytesReadArr[slot] = std::min((off_t)events[i].res, fileSize -
writeOffset);

                //меняем буферы слота местами без копирования:
                std::swap(readBufs[slot], writeBufs[slot]);

                //toWrite тоже должен быть кратен ALIGN для O_DIRECT,
округляем вверх
                size_t toWrite = ((bytesReadArr[slot] + ALIGN - 1) / ALIGN) *
ALIGN;

                //заполняет структуру iocb для операции записи count байт из
buf в fd начиная со смещения offset
                io_prep_pwrite(&writeCbs[slot], dstFd, writeBufs[slot],
toWrite, writeOffset);

                //сохраняем индекс слота чтобы при завершении записи, чтобы
знать какой слот освободился
                writeCbs[slot].data = (void*)(intptr_t)slot;
                writeOffset += bytesReadArr[slot];

                int toSubmit = 0;
                cbs[toSubmit++] = &writeCbs[slot]; //запись

                //если файл не кончился - сразу читаем следующий блок в этот
же слот
                if(readOffset < alignedSize){
                    size_t toRead = std::min((off_t)blockSize, alignedSize -
readOffset);
                    io_prep_pread(&readCbs[slot], srcFd, readBufs[slot],
toRead, readOffset);
                    readCbs[slot].data = (void*)(intptr_t)slot;
                    cbs[toSubmit++] = &readCbs[slot];
                    readOffset += toRead;
                }
```

```
                //отправляем обе операции одним вызовом io_submit
                rc = io_submit(ctx, toSubmit, cbs);
                if(rc < 0)
                    std::cout << "io_submit failed: " << strerror(-rc) <<
std::endl;
                else
                    inFlight += toSubmit;

            }
        }
    }

    //Очистка
    //io_destroy освобождает контекст AIO и все связанные ресурсы ядра
    //важно вызывать после того как все операции завершены
    io_destroy(ctx);

    for(int i = 0; i < overlapCount; i++){
        free(readBufs[i]);
        free(writeBufs[i]);
    }
    close(srcFd);
    close(dstFd);

    //обрезаем файл до реального размера - убираем нулевой padding
    truncate(destPath.c_str(), fileSize);

    return true;
}
```

11) TestCopying.hpp

```
#pragma once
#include <chrono>
#include <fstream>
#include "FileCopier.hpp"

class TestCopying
{
private:
    int numTests;
    size_t fileSize;
    FileCopier fileCpy;
    void createTestFile(const std::string& filename);
    void createFilesForTest();
    void doCopying();
public:
    TestCopying();
    ~TestCopying();
    int doOneCycleTest();
```

```
    void doTests(const int _numTests, const FileCopier& _fileCpy, const size_t
_fileSize, const int numCycles);
    void deleteAllFilesInDir(const std::string& dir);
};
```

12) TestCopying.cpp

```
#include "TestCopying.hpp"

TestCopying::TestCopying(){
    numTests=0;
    fileSize=0;
}
TestCopying::~TestCopying(){}

void TestCopying::doTests(const int _numTests, const FileCopier& _fileCpy,
const size_t _fileSize, const int numCycles){
    numTests=_numTests;
    fileCpy=_fileCpy;
    fileSize=_fileSize*1024;
    double totalRuntime, avrgRuntime;
    std::cout << "Тестирование...\n";
    for(int i=0;i<numCycles;i++){
        totalRuntime+=static_cast<double>(doOneCycleTest());
    }
    avrgRuntime = totalRuntime/numCycles;
    std::cout << "Среднее время выполнения (" << numCycles << " циклов): " <<
avrgRuntime <<" мс"<< std::endl;
}

int TestCopying::doOneCycleTest(){
    if(!std::filesystem::create_directories("filesOrig")){
        deleteAllFilesInDir("filesOrig");
    }
    if(!std::filesystem::create_directories("filesCopy")){
        deleteAllFilesInDir("filesCopy");
    }
    createFilesForTest();
    unsigned int runtime;
    auto start = std::chrono::high_resolution_clock::now();
    doCopying();
    auto end = std::chrono::high_resolution_clock::now();
    runtime = std::chrono::duration_cast<std::chrono::milliseconds>(end -
start).count();
    return runtime;
}

void TestCopying::deleteAllFilesInDir(const std::string& dst){
    for(const auto& entry : std::filesystem::directory_iterator(dst)){
        if(std::filesystem::is_regular_file(entry.status())){
```

```
            std::filesystem::remove(entry.path());
        }
    }
}

void TestCopying::createFilesForTest(){
    for(int i=0;i<numTests;i++){
        createTestFile(std::to_string(i) + ".txt");
    }
}

void TestCopying::createTestFile(const std::string& filename){
    std::vector<char> buffer(fileSize, 0);
    std::ofstream file("filesOrig/"+filename);

    if(file.is_open()){
        for(size_t i=0; i<fileSize;i++){
            file << '0';
        }
        file.close();
    } else{
        std::cout << "Ошибка: не удалось создать файл " << filename <<
std::endl;
    }
}

void TestCopying::doCopying(){
    for(const auto& entry : std::filesystem::directory_iterator("filesOrig")){
        if(std::filesystem::is_regular_file(entry.status())){
            fileCpy.copyFile(entry.path(), "filesCopy");
        }
    }
}
```

13) interface.hpp

```
#pragma once
#include "FileCopier.hpp"
#include "TestCopying.hpp"

class Interface
{
private:
    FileCopier fileCpy;
    TestCopying cpyTester;
    void runFileCopying();
    void runCopyTests();
    void runChangingAlign();
    void runChangingOverlapCount();
    void runFileDeleting();
```

```
    void runBlockSizeAutoTest(const int _numTests, const size_t _fileSize,
const int numCycles);
    void runOLOAutoTest(const int _numTests, const size_t _fileSize, const int
numCycles);
    void showParameteres();
    void showMenu();
public:
    Interface();
    ~Interface();
    void show();
};
```

14)interface.cpp

```
#include "interface.hpp"

Interface::Interface(){
    system("clear");
    runChangingAlign();
    runChangingOverlapCount();
}
Interface::~Interface(){}

void Interface::show(){
    bool running=true;
    int userChoice;
    bool wrongOption=false;
    while(running){
        system("clear");
        if(wrongOption){
            std::cout << "WRONG OPTION!\nTRY AGAIN!\n";
            wrongOption=false;
        }
        showParameteres();
        showMenu();
        std::cout<<"Choose option: ";
        std::cin >> userChoice;
        system("clear");
        switch (userChoice)
        {
        case 0:
            running=false;
            std::cout << "=======EXITING=======\n";
            std::cout << "Exiting...\n";
            break;
        case 1:
            showParameteres();
            std::cout << "==========COPY FILE==========\n";
            runFileCopying();
```

```
            break;
        case 2:
            showParameteres();
            std::cout << "==========COPY TESTS==========\n";
            runCopyTests();
            break;
        case 3:
            showParameteres();
            std::cout << "==========CHANGE ALIGN==========\n";
            runChangingAlign();
            break;
        case 4:
            showParameteres();
            std::cout << "==========CHANGE OVERLAPPING OPS==========\n";
            runChangingOverlapCount();
            break;
        case 5:
            std::cout << "==========FILES DELETING==========\n";
            runFileDeleting();
            break;
        default:
            wrongOption=true;
            break;
        }
        if(!wrongOption){
            std::cout << "Press ENTER to continue...\n";
            std::cin.ignore();
            std::cin.get();
        }
    }
}

void Interface::runCopyTests(){
    int numTests, numCycles;
    size_t fileSize;
    char userChoice;
    std::cout << "Введите количество файлов для копирования: ";
    std::cin >> numTests;
    std::cout << "Введите размер создаваемых файлов (в килобайтах): ";
    std::cin >> fileSize;
    std::cout << "Введите количество прогонов: ";
    std::cin >> numCycles;
    std::cout << "Прогнать автотест? (y/n): ";
    std::cin >> userChoice;
    if(userChoice=='y'){
        std::cout << "Прогнать автотест размера блока, или количества
перекрывающих операций (введите 1 или 2): ";
        std::cin >> userChoice;
```

```
        if(userChoice=='1'){
            runBlockSizeAutoTest(numTests, fileSize, numCycles);
        }
        else if(userChoice=='2'){
            runOLOAutoTest(numTests, fileSize, numCycles);
        }
    }
    else{
        cpyTester.doTests(numTests, fileCpy, fileSize, numCycles);
    }
    std::cout << "Тесты завершены\n";
}

void Interface::runBlockSizeAutoTest(const int _numTests, const size_t
_fileSize, const int numCycles){
    int maxMultipl;
    std::cout << "Введите максимальный блок копирования (в кб) (шаг +2): ";
    std::cin >> maxMultipl;
    for(int i=1;i<=maxMultipl;i+=2){
        fileCpy.setAlign(1, i);
        std::cout << "Размер блока: " << i << " кб\n";
        if(i==1){
            i--;
        }
        cpyTester.doTests(_numTests, fileCpy, _fileSize, numCycles);
        std::cout << std::endl;
    }
}

void Interface::runOLOAutoTest(const int _numTests, const size_t _fileSize,
const int numCycles){
    int maxOLO;
    std::cout << "Введите максимальное количество перекрывающих операций (шаг
+2): ";
    std::cin >> maxOLO;
    for(int i=1;i<=maxOLO;i+=2){
        fileCpy.setOverlapCount(i);
        std::cout << "Количество перекрывающих операций: " << i << '\n';
        if(i==1){
            i--;
        }
        cpyTester.doTests(_numTests, fileCpy, _fileSize, numCycles);
        std::cout << std::endl;
    }
}

void Interface::runFileCopying(){
    std::string destDir, srcPath;
```

```
    std::cout << "Введите путь к файлу, который копировать: ";
    std::cin >> srcPath;
    std::cout << "Введите директорию, куда копировать файл: ";
    std::cin>>destDir;
    if(fileCpy.copyFile(srcPath, destDir)){
        std::cout << "Копирование успешно завершено\n";
    }
    else{
        std::cout << "Произошла ошибка!\n";
    }
}

void Interface::runChangingAlign(){
    int Align, blockSizeMltpl;
    std::cout << "Введите выравнивание буфера в килобайтах: ";
    std::cin >> Align;
    std::cout << "Введите множитель для блока копирования (во сколько раз
больше выравнивания): ";
    std::cin >> blockSizeMltpl;
    fileCpy.setAlign(Align, blockSizeMltpl);
}

void Interface::runChangingOverlapCount(){
    int overlapCount;
    std::cout << "Введите число перекрывающих операций ввода и вывода: ";
    std::cin >> overlapCount;
    fileCpy.setOverlapCount(overlapCount);
}

void Interface::runFileDeleting(){
    if(std::filesystem::exists("filesOrig")){
        if(std::filesystem::is_directory("filesOrig")){
            cpyTester.deleteAllFilesInDir("filesOrig");
        }
    }

    if(std::filesystem::exists("filesCopy")){
        if(std::filesystem::is_directory("filesCopy")){
            cpyTester.deleteAllFilesInDir("filesCopy");
        }
    }
    std::cout << "Files deleted\n";
}

void Interface::showParameteres(){
    std::cout<<"=====Parameters=====\n";
    std::cout<<"ALIGN: "<<fileCpy.getAlign()<<std::endl;
    std::cout<<"Block size: "<<fileCpy.getBlockSize()<<std::endl;
```

```
    std::cout<<"Number of overlapping operations:
"<<fileCpy.getOverlapCount()<<std::endl;
}

void Interface::showMenu(){
    std::cout<<"=====Menu=====\n";
    std::cout<<"0.  Exit\n";
    std::cout<<"1.  Copy file in directory"<<std::endl;
    std::cout<<"2.  Do copy tests" << std::endl;
    std::cout<<"3.  Change buffer align" << std::endl;
    std::cout<<"4.  Change number of overlapping operations" << std::endl;
    std::cout<<"5.  Delete all files in directories"<<std::endl;
}
```

15) main.cpp

```
#include "interface.hpp"

int main()
{
    Interface interface;
    interface.show();
    return 0;
}
```